

端對端遷移：從分片的內部部署 MySQL 遷移至 Cloud Spanner (GoogleSQL)

來源：<https://codelabs.developers.google.com/sharded-sql-to-spanner-migration?hl=zh-tw>

步驟數：12

目錄

1. [事前準備](#)
2. [瞭解遷移程序](#)
3. [環境設定](#)
4. [設定來源 MySQL 資料庫](#)
5. [設定 Cloud Spanner](#)
6. [使用 Spanner 遷移工具 \(SMT\) 轉換結構定義](#)
7. [初始化變更資料擷取 \(CDC\)](#)
8. [自訂轉換](#)
9. [從 MySQL 大量遷移資料至 Spanner](#)
10. [開始即時遷移 \(CDC\)](#)
11. [設定反向複製 \(Spanner 至 MySQL\)](#)
12. [清除資源](#)

1. 事前準備

本程式碼研究室將逐步說明如何將分片的內部部署 MySQL 資料庫，遷移至使用 GoogleSQL 方言的 Cloud Spanner 資料庫。您將使用 Google Cloud 服務，包括 Spanner 移轉工具 (SMT)、Dataflow、Datastream、PubSub 和 Google Cloud Storage。

報告內容：

- 什麼是分片環境，以及如何設定。
- 說明如何使用 Spanner 遷移工具 (SMT) 網頁介面，將 MySQL 結構定義轉換為與 Spanner 相容的結構定義，並進行進階結構定義修改。
- 說明如何使用 Dataflow，從分片 MySQL 執行個體大量遷移資料至 Cloud Spanner。
- 說明如何使用 Datastream 和 Dataflow，從分片 MySQL 執行個體設定持續複製 (CDC) 至 Cloud Spanner。
- 如何設定從 Spanner 反向複製回已分片的 MySQL 執行個體。
- 瞭解如何使用自訂轉換，在大量、即時和反向遷移期間填入額外欄位。
- 如何使用主鍵設定分片轉換。

本程式碼研究室不會涵蓋以下主題：

- 進階自訂網路。
- 從頭開始建構自訂 Dataflow 範本。
- 調整遷移作業效能。
- **應用程式遷移**：本程式碼研究室著重於資料庫層 (結構定義和資料)。不包含重新部署或遷移應用程式服務的作業程序。

注意：雖然本程式碼研究室會處理資料和結構定義，但您仍需**手動修改應用程式程式碼** (包括更新資料庫驅動程式、連線邏輯等)，才能完成端對端遷移作業。

軟硬體需求

- 已啟用計費功能的 Google Cloud 專案。
- 具備足夠的 [IAM 權限](#)，可啟用 API，以及建立/管理 Spanner、Dataflow、Datastream 和 GCS 資源。雖然專案 `Owner` 角色最適合程式碼研究室，但「環境設定」會涵蓋更具體的角色。
- 在設定階段，我們會佈建小型 Compute Engine VM，模擬內部部署伺服器。確認專案配額允許建立 VM。
- 網路瀏覽器，例如 Google Chrome。
- 基本熟悉 Google Cloud 控制台和 `gcloud` 等指令列工具。
- 存取殼層環境。建議使用 [Cloud Shell](#)，因為當中已包含 `gcloud`。

如要進一步瞭解上述設定，請參閱「[環境設定](#)」一節。

2. 瞭解遷移程序

遷移已分割的資料庫時，需要將多個實體和邏輯 MySQL 執行個體整合成單一的 Spanner 資料庫，並進行水平擴充。本節將說明遷移作業所用的架構和主要工具。

遷移流程架構

遷移程序包括以下階段：

1. 結構定義轉換：

- **用途：**將來源資料庫結構定義轉換為相容的 Cloud Spanner 結構定義。
- **工具：**Spanner 遷移工具 (SMT)
- **程序：**SMT 會分析來源資料庫結構定義，並產生對等的 Spanner 資料定義語言 (DDL)。系統會在目標 Spanner 執行個體中建立資料庫，然後自動套用 DDL。

2. 大量資料遷移：

- **用途：**從來源資料庫將現有資料完整載入至已佈建的 Spanner 資料表。
- **工具：**Dataflow，使用 Google 提供的 `Sourcedb to Spanner` 範本。
- **程序：**這項 Dataflow 工作會讀取指定來源資料表的所有資料，並寫入對應的 Spanner 資料表。這項操作會在建立 Spanner 結構定義後進行。

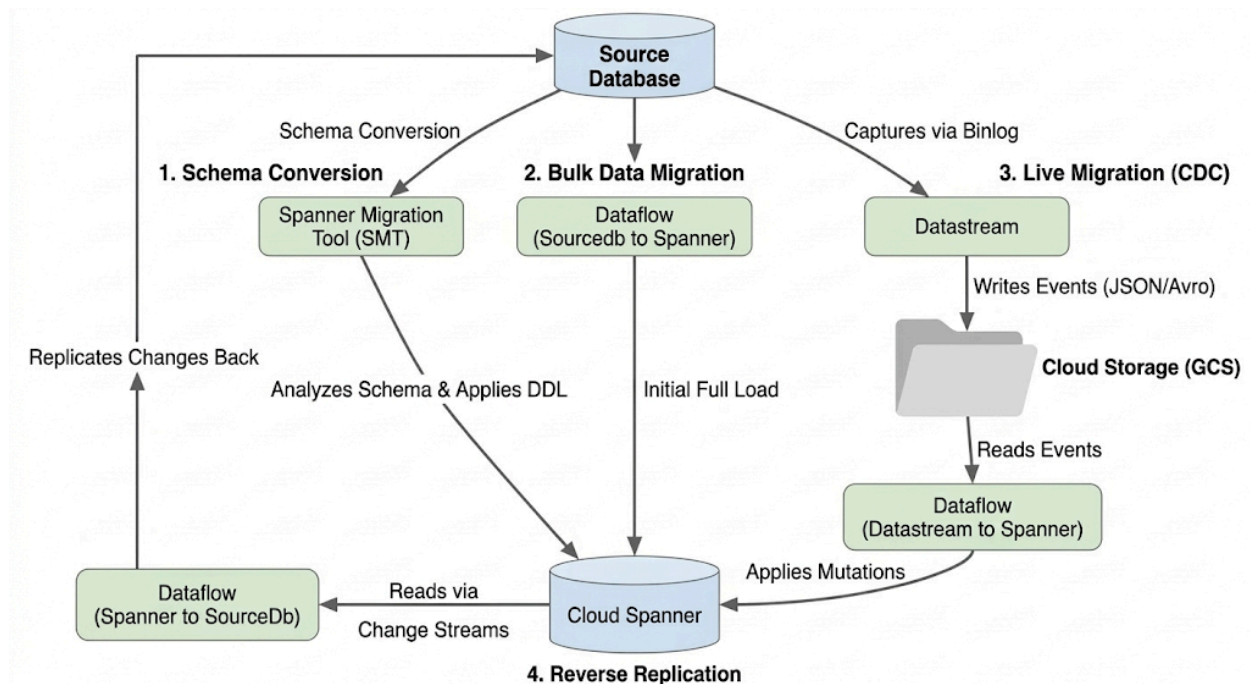
3. 即時遷移 (CDC)：

- **用途：**擷取來源資料庫的持續變更，並以近乎即時的方式套用至 Cloud Spanner，盡量減少遷移期間的停機時間。
- **工具：**
 - **Datastream：**擷取來源資料庫中的變更 (插入、更新、刪除)，並寫入 Cloud Storage (GCS)。
 - **Dataflow：**使用 `Datastream to Spanner` 範本從 GCS 讀取變更事件，並套用至 Cloud Spanner。

4. 反向複製：

- **用途：**將 Cloud Spanner 的資料變更複製回來源資料庫。這項功能可用於備援策略、分階段遷移，或在來源中維護副本，以因應特定用途。
- **工具：**Dataflow，使用 `Spanner to SourceDb` 範本。
- **處理程序：**這項工作會使用 Spanner 變更串流擷取 Spanner 中的修改內容，並將這些內容寫回來源資料庫執行個體。

下圖說明元件和資料流程：



重要術語：

- **實體分片**：代管資料庫的實際基礎伺服器或運算執行個體 (在本例中為模擬的**地端部署** GCE VM)。
- **邏輯分片**：實體伺服器中的個別資料庫結構定義。
- **Compute Engine (GCE) VM**：託管於 Google Cloud 雲端基礎架構的虛擬機器。在本程式碼研究室中，我們使用 GCE VM 模擬獨立的「地端」裸機伺服器，做為來源 MySQL 資料庫的主機。
- **Spanner 遷移工具 (SMT)**：這項工具可用於評估 MySQL 結構定義、建議對應的 Spanner 結構定義，以及產生 Spanner 資料定義語言 (DDL)。
- **資料定義語言 (DDL)**：用於定義及修改資料庫結構的陳述式，例如 `CREATE TABLE` 陳述式。SMT 會根據 Cloud SQL 結構定義產生 Spanner DDL。
- **Dataflow**：全代管的無伺服器資料處理服務。在本程式碼研究室中，這項工具用於執行 Google 提供的範本，以進行大量資料移轉、套用 Datastream 變更，以及反向複製。
- **Datastream**：無伺服器變更資料擷取 (CDC) 和複製服務。在本程式碼研究室中，這項工具用於將本機託管的 MySQL 執行個體中的變更串流至 Cloud Storage。
- **Spanner 變更串流**：Spanner 功能，可即時串流輸出資料變更 (插入、更新、刪除)，做為反向複製的來源。
- **Pub/Sub**：訊息服務，用於分離產生事件的服務與處理事件的服務。在本程式碼研究室中，每當 Datastream 將新的變更檔案上傳至 Cloud Storage 時，就會觸發 Dataflow 處理更新。

步驟 3 · 約 5 分鐘

3. 環境設定

開始遷移前，您需要設定 Google Cloud 雲端專案並啟用必要服務。

1. 選取或建立 Google Cloud 專案

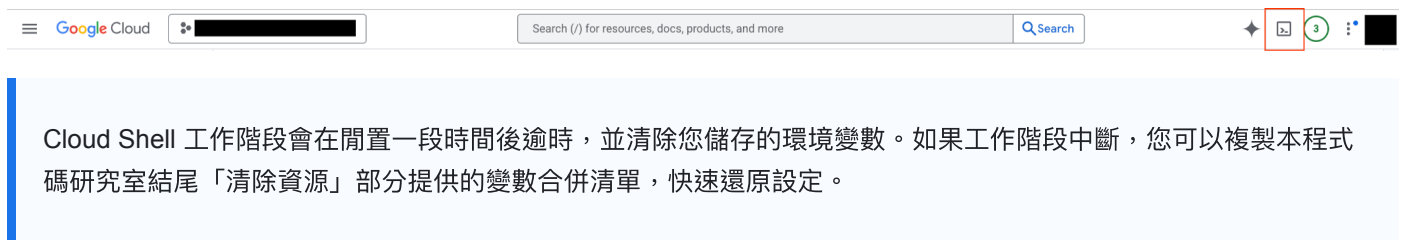
您必須擁有已啟用計費功能的 Google Cloud 專案，才能使用本程式碼研究室中的服務。

1. 在 Google Cloud 控制台中，前往專案選取器頁面：[前往專案選取器](#)
2. 選取或建立 Google Cloud 專案。
3. 請確認您已為專案啟用計費功能。瞭解如何[確認專案已啟用計費功能](#)。

2. 開啟 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入 `gcloud` CLI 和其他必要工具。

- 按一下 Google Cloud 控制台右上角的「啟用 Cloud Shell」按鈕。
- 系統會在主控台底部的新頁框中開啟 Cloud Shell 工作階段，並顯示指令列提示。



3. 設定專案和环境變數

在 Cloud Shell 中，為專案 ID 和您要使用的區域設定一些環境變數。

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1" # Or your preferred region
export ZONE="us-central1-a" # Or a zone within your selected region

gcloud config set project $PROJECT_ID
gcloud config set compute/region $REGION
gcloud config set compute/zone $ZONE

echo "Project ID: $PROJECT_ID"
echo "Region: $REGION"
echo "Zone: $ZONE"
```

重要事項：請為本程式碼研究室中建立的所有資源 (Cloud Spanner、Datastream、Dataflow) 使用相同區域，盡量減少延遲和網路複雜度。

4. 啟用必要的 Google Cloud API

啟用 Cloud Spanner、Dataflow、Datastream 和其他相關服務所需的 API。

```
gcloud services enable \  
  spanner.googleapis.com \  
  dataflow.googleapis.com \  
  datastream.googleapis.com \  
  pubsub.googleapis.com \  
  storage.googleapis.com \  
  compute.googleapis.com \  
  sqladmin.googleapis.com \  
  servicenetworking.googleapis.com \  
  cloudresourcemanager.googleapis.com
```

這個指令可能需要幾分鐘才能完成。

步驟 4 · 約 20 分鐘

4. 設定來源 MySQL 資料庫

在本節中，我們將佈建兩部 Compute Engine 虛擬機器 (即 2 個「實體分片」)，模擬內部部署的分片 MySQL 架構。接著在兩部 VM 上安裝 MySQL，並在每部 VM 上建立兩個資料庫 (即「邏輯分片」)。

1. 建立 Compute Engine VM (實體分片)

在 Cloud Shell 中執行下列指令，建立兩個搭載 Ubuntu 的 VM。稍後我們會指派網路標記，允許 MySQL 傳入流量。

```
# Create Physical Shard 1
gcloud compute instances create mysql-physical-1 \
  --zone=$ZONE \
  --machine-type=e2-small \
  --image-family=ubuntu-2204-lts \
  --image-project=ubuntu-os-cloud \
  --tags=mysql-server

# Create Physical Shard 2
gcloud compute instances create mysql-physical-2 \
  --zone=$ZONE \
  --machine-type=e2-small \
  --image-family=ubuntu-2204-lts \
  --image-project=ubuntu-os-cloud \
  --tags=mysql-server
```

2. 設定防火牆規則

如要允許安全 SSH 存取權，但不要公開，並啟用 Datastream 連線，請按照下列步驟操作：

透過 IAP 建立 SSH 的防火牆規則：

這項規則允許 Identity-Aware Proxy 透過 SSH 通訊埠 (22) 連線至 VM。

```
gcloud compute firewall-rules create allow-ssh-iap \
  --direction=INGRESS \
  --priority=1000 \
  --network=default \
  --action=ALLOW \
  --rules=tcp:22 \
  --source-ranges=35.235.240.0/20 \
  --target-tags=mysql-server
```

為 Datastream (MySQL 連接埠) 建立防火牆規則：

Datastream 必須能夠透過標準 MySQL 通訊埠 (3306) 連線至這些 VM。

```
gcloud compute firewall-rules create allow-mysql-datastream \
  --direction=INGRESS \
  --priority=1000 \
  --network=default \
  --action=ALLOW \
  --rules=tcp:3306 \
  --source-ranges=0.0.0.0/0 \
  --target-tags=mysql-server
```

0.0.0.0/0 不建議用於正式環境，因為這會將資料庫暴露於公開網際網路。這裡這麼做只是為了簡化程式碼研究室。在實際情境中，您應限制資料庫存取權：

- **Datastream**：請只授權您所在區域的 [Datastream 公開 IP 位址](#)，而非 0.0.0.0/0。
- **Dataflow**：為避免將 Cloud SQL 暴露給 Dataflow 工作站的 0.0.0.0/0，請使用 [Cloud SQL 私人 IP](#)，並將 Dataflow 設定為使用相同的虛擬私有雲網路。
- 如要從任何環境建立安全連線並通過 IAM 授權，請使用 [Cloud SQL 驗證 Proxy](#)。

3. 在實體分片 1 上安裝及設定 MySQL

透過 SSH 連線至第一個 VM，安裝 MySQL 並設定二進位記錄 (Datastream 必須使用這項功能才能進行即時複製)。

1. 透過 SSH 連線至第一個 VM：

```
gcloud compute ssh mysql-physical-1 --zone=$ZONE --tunnel-through-iap
```

--tunnel-through-iap 旗標會透過 Google 的 Identity-Aware Proxy (IAP) 轉送 SSH 連線。這是安全防護的最佳做法，因為您可以在不將 SSH 通訊埠 (通訊埠 22) 暴露於公用網際網路的情況下，安全地連線至 VM。

2. 安裝 MySQL：

```
sudo apt-get update
sudo apt-get install mysql-server-8.0 -y

# Verify the installation and version
sudo mysql --version
```

3. 設定 `mysqld.cnf` 檔案，啟用二進位記錄功能並允許外部連線：

```
sudo sed -i 's/bind-address.*/bind-address = 0.0.0.0/' /etc/mysql/mysql.conf.d/mysqld.cnf
echo -e "[mysqld]\nserver-id=1\nlog_bin=/var/log/mysql/mysql-bin.log\nbinlog_format=ROW" |
sudo tee -a /etc/mysql/mysql.conf.d/mysqld.cnf
```

4. 重新啟動 MySQL 以套用變更：

```
sudo systemctl restart mysql
```

4. 建立邏輯分片、插入資料，以及建立 Datastream 使用者 (分片 1)

透過 SSH 連線至 `mysql-physical-1` 時，請登入 MySQL 提示：

```
sudo mysql
```

執行下列 SQL 指令。這項指令碼會建立兩個不同的邏輯分片 (`shard0_db` 和 `shard1_db`)，在兩者中設定相同的結構定義，將可明確識別身分的資料插入每個分片 (用來示範分片)，並為 Datastream 建立複製使用者。

執行下列 SQL 指令，建立前兩個邏輯分片、資料表，以及 Datastream 的複製使用者：

```

CREATE DATABASE shard0_db;
CREATE DATABASE shard1_db;

USE shard0_db;
CREATE TABLE Customers (
    CustomerId INT NOT NULL,
    CustomerName VARCHAR(255),
    CreditLimit DECIMAL(10, 2) NOT NULL,
    LegacyRegion VARCHAR(50), -- Renamed to LoyaltyTier in Spanner

    PRIMARY KEY (CustomerId),
    CONSTRAINT CHK_CreditLimit CHECK (CreditLimit > 1000) -- Relaxed in Spanner to > 0
);

CREATE TABLE Orders (
    CustomerId INT NOT NULL, -- Logically references Customers.CustomerId in Spanner
    OrderId INT NOT NULL,
    OrderValue DECIMAL(10, 2),
    LegacyOrderSystem VARCHAR(50), -- Extra column in Source, to be dropped in Spanner

    PRIMARY KEY (CustomerId, OrderId) -- Spanner PK will have one additional column in PK
);

INSERT INTO Customers (CustomerId, CustomerName, CreditLimit, LegacyRegion) VALUES
(4, 'David E.', 2000.00, 'EAST'),
(8, 'Eleanor F.', 8100.00, 'WEST'),
(12, 'Frank G.', 12000.00, 'NORTH'),
(16, 'Grace H.', 6500.00, 'SOUTH');

INSERT INTO Orders (CustomerId, OrderId, OrderValue, LegacyOrderSystem) VALUES
(4, 101, 150.00, 'WebStore_v1'),
(4, 102, 25.50, 'InStore_POS'),
(8, 103, 75.00, 'MobileApp_Legacy'),
(12, 104, 3000.00, 'WebStore_v1'),
(16, 105, 120.00, 'Partner_API');

USE shard1_db;
CREATE TABLE Customers (
    CustomerId INT NOT NULL,
    CustomerName VARCHAR(255),
    CreditLimit DECIMAL(10, 2) NOT NULL,
    LegacyRegion VARCHAR(50), -- Renamed to LoyaltyTier in Spanner
    PRIMARY KEY (CustomerId),
    CONSTRAINT CHK_CreditLimit CHECK (CreditLimit > 1000) -- Relaxed in Spanner to > 0
);

CREATE TABLE Orders (
    CustomerId INT NOT NULL, -- Logically references Customers.CustomerId in Spanner
    OrderId INT NOT NULL,
    OrderValue DECIMAL(10, 2),
    LegacyOrderSystem VARCHAR(50), -- Extra column in Source, to be dropped in Spanner
    PRIMARY KEY (CustomerId, OrderId) -- Spanner PK will have one additional column in PK

```

```
);

INSERT INTO Customers (CustomerId, CustomerName, CreditLimit, LegacyRegion) VALUES
(1, 'Agnes N.', 5100.00, 'NORTHEAST'), (5, 'Alice I.', 15000.00, 'EAST'),
(9, 'Bob J.', 7500.00, 'WEST'),
(13, 'Charlie K.', 2200.00, 'CENTRAL');

INSERT INTO Orders (CustomerId, OrderId, OrderValue, LegacyOrderSystem) VALUES
(1, 201, 50.00, 'MobileApp_Legacy'),
(5, 202, 1250.00, 'WebStore_v1'),
(5, 203, 80.00, 'Partner_API'),
(9, 204, 600.00, 'InStore_POS'),
(13, 205, 199.99, 'WebStore_v1');

-- Create Datastream Replication User
CREATE USER 'datastream_user'@'%' IDENTIFIED BY 'complex_password_123';
GRANT REPLICATION SLAVE, REPLICATION CLIENT, SELECT, INSERT, UPDATE, DELETE ON *.* TO
'datastream_user'@'%';
FLUSH PRIVILEGES;
```

如要查看上述架構的 dump 檔案，請按[這裡](#)。請務必另外建立資料串流複製使用者，因為傾印檔案中不包含這項使用者。

5. 驗證資料

快速檢查資料是否已存在：

```
SELECT 'Customers shard0_db' AS tbl, COUNT(*) FROM shard0_db.Customers
UNION ALL
SELECT 'Orders shard0_db', COUNT(*) FROM shard0_db.Orders
UNION ALL
SELECT 'Customers shard1_db', COUNT(*) FROM shard1_db.Customers
UNION ALL
SELECT 'Orders shard1_db', COUNT(*) FROM shard1_db.Orders;
EXIT;
```

預期輸出內容：

```
+-----+-----+
| tbl                | COUNT(*) |
+-----+-----+
| Customers shard0_db |         4 |
| Orders shard0_db    |         5 |
| Customers shard1_db |         4 |
| Orders shard1_db    |         5 |
+-----+-----+
```

輸入 `exit`，結束與實體分片 1 VM 的連線。

6. 針對實體分片 2 重複上述步驟

現在，您將對第二個 VM 重複執行完全相同的程序，但會建立 `shard2_db` 和 `shard3_db`，並變更 `server-id`。

1. 透過 SSH 連線至第二部 VM：

```
gcloud compute ssh mysql-physical-2 --zone=$ZONE --tunnel-through-iap
```

2. 安裝 MySQL：

```
sudo apt-get update
sudo apt-get install mysql-server-8.0 -y
```

3. 設定 `mysqld.cnf` 檔案，啟用二進位檔記錄並允許外部連線 (請注意，伺服器 ID 必須不同，例如 2)

```
sudo sed -i 's/bind-address.*/bind-address = 0.0.0.0/' /etc/mysql/mysql.conf.d/mysqld.cnf

echo -e "[mysqld]\nserver-id=2\nlog_bin=/var/log/mysql/mysql-bin.log\nbinlog_format=ROW" |
sudo tee -a /etc/mysql/mysql.conf.d/mysqld.cnf
```

4. 重新啟動 MySQL 以套用變更：

```
sudo systemctl restart mysql
```

5. 輸入 MySQL (`sudo mysql`)，然後執行步驟 4 中稍作修改的 SQL：

```

CREATE DATABASE shard2_db;
CREATE DATABASE shard3_db;

USE shard2_db;
CREATE TABLE Customers (
    CustomerId INT NOT NULL,
    CustomerName VARCHAR(255),
    CreditLimit DECIMAL(10, 2) NOT NULL,
    LegacyRegion VARCHAR(50), -- Renamed to LoyaltyTier in Spanner
    PRIMARY KEY (CustomerId),
    CONSTRAINT CHK_CreditLimit CHECK (CreditLimit > 1000) -- Relaxed in Spanner to > 0
);

CREATE TABLE Orders (
    CustomerId INT NOT NULL, -- Logically references Customers.CustomerId in Spanner
    OrderId INT NOT NULL,
    OrderValue DECIMAL(10, 2),
    LegacyOrderSystem VARCHAR(50), -- Extra column in Source, to be dropped in Spanner
    PRIMARY KEY (CustomerId, OrderId) -- Spanner PK will have one additional column in PK
);

INSERT INTO Customers (CustomerId, CustomerName, CreditLimit, LegacyRegion) VALUES
(2, 'Brian K.', 2500.00, 'SOUTHWEST'),
(6, 'Diana L.', 1999.00, 'NORTH'),
(10, 'Edward M.', 11000.00, 'EAST'),
(14, 'Fiona N.', 3000.00, 'WEST');

INSERT INTO Orders (CustomerId, OrderId, OrderValue, LegacyOrderSystem) VALUES
(2, 301, 100.00, 'CallCenter_System'),
(6, 302, 99.00, 'MobileApp_Legacy'),
(10, 303, 1000.00, 'WebStore_v1'),
(10, 304, 2500.00, 'InStore_POS'),
(14, 305, 130.00, 'MobileApp_Legacy');

USE shard3_db;
CREATE TABLE Customers (
    CustomerId INT NOT NULL,
    CustomerName VARCHAR(255),
    CreditLimit DECIMAL(10, 2) NOT NULL,
    LegacyRegion VARCHAR(50), -- Renamed to LoyaltyTier in Spanner
    PRIMARY KEY (CustomerId),
    CONSTRAINT CHK_CreditLimit CHECK (CreditLimit > 1000) -- Relaxed in Spanner to > 0
);

CREATE TABLE Orders (
    CustomerId INT NOT NULL, -- Logically references Customers.CustomerId in Spanner
    OrderId INT NOT NULL,
    OrderValue DECIMAL(10, 2),
    LegacyOrderSystem VARCHAR(50), -- Extra column in Source, to be dropped in Spanner
    PRIMARY KEY (CustomerId, OrderId) -- Spanner PK will have one additional column in PK
);

```

```

INSERT INTO Customers (CustomerId, CustomerName, CreditLimit, LegacyRegion) VALUES
(3, 'Cathy Z.', 6000.00, 'CENTRAL'),
(7, 'George O.', 18000.00, 'SOUTH'),
(11, 'Helen P.', 4000.00, 'NORTHEAST'),
(15, 'Ivy Q.', 9500.00, 'SOUTHWEST');

INSERT INTO Orders (CustomerId, OrderId, OrderValue, LegacyOrderSystem) VALUES
(3, 401, 600.00, 'InStore_POS'),
(7, 402, 1200.00, 'CallCenter_System'),
(11, 403, 350.00, 'MobileApp_Legacy'),
(15, 404, 800.00, 'WebStore_v1'),
(99, 999, 25.00, 'CallCenter_System'); -- Failure row during Bulk Migration due to violation
of interleaving

-- Create Datastream Replication User
CREATE USER 'datastream_user'@'%' IDENTIFIED BY 'complex_password_123';
GRANT REPLICATION SLAVE, REPLICATION CLIENT, SELECT, INSERT, UPDATE, DELETE ON *.* TO
'datastream_user'@'%;
FLUSH PRIVILEGES;

-- Verify Data
SELECT 'Customers shard2_db' AS tbl, COUNT(*) FROM shard2_db.Customers
UNION ALL
SELECT 'Orders shard2_db', COUNT(*) FROM shard2_db.Orders
UNION ALL
SELECT 'Customers shard3_db', COUNT(*) FROM shard3_db.Customers
UNION ALL
SELECT 'Orders shard3_db', COUNT(*) FROM shard3_db.Orders;

EXIT;

```

預期輸出內容：

```

+-----+-----+
| tbl           | COUNT(*) |
+-----+-----+
| Customers shard2_db |      4 |
| Orders shard2_db   |      5 |
| Customers shard3_db |      4 |
| Orders shard3_db   |      5 |
+-----+-----+

```

如要查看上述架構的 dump 檔案，請按[這裡](#)。請務必另外建立資料串流複製使用者，因為傾印檔案中不包含這項使用者。

輸入 `exit` 即可結束與 VM 的連線。

步驟 5 · 約 5 分鐘

5. 設定 Cloud Spanner

接下來，請設定要遷移資料的目標 Cloud Spanner 執行個體。

1. 建立 Cloud Spanner 執行個體

在與 **Compute Engine VM** 相同的區域建立 Cloud Spanner 執行個體，盡量減少延遲。這個指令會建立適合本程式碼實驗室的小型執行個體，並使用 100 個處理單元。

```
export SPANNER_INSTANCE_NAME="target-spanner-instance"
export SPANNER_DATABASE_NAME="sharded-target-db"
export SPANNER_CONFIG="regional-${REGION}"

gcloud spanner instances create $SPANNER_INSTANCE_NAME \
  --config=$SPANNER_CONFIG \
  --description="Target Spanner Instance" \
  --processing-units=100
```

建立執行個體可能需要一到兩分鐘。

雖然我們在 MySQL 中建立了四個獨立的邏輯資料庫 (分片)，但只需要建立一個 Cloud Spanner 執行個體和資料庫。Spanner 將做為所有分片資料的整合式目標，並可水平擴充。

6. 使用 Spanner 遷移工具 (SMT) 轉換結構定義

使用 Spanner 遷移工具 (SMT) 網頁版 UI 連線至其中一個邏輯分片 (`shard0_db`)，分析其結構定義，並套用多項進階修改，然後再轉換為 Cloud Spanner。

1. 安裝 SMT

我們將直接從 Cloud Shell 執行 SMT 網頁介面。在 Cloud Shell 終端機中，下載並解壓縮最新版 SMT：

```
sudo apt-get update && sudo apt-get install google-cloud-cli-spanner-migration-tool

# Verify installation
gcloud alpha spanner migrate web --help
```

這些指令僅適用於 Linux。如果是 Windows/Mac，請按照[這些步驟](#)從來源建構。

2. 連線至來源資料庫

1. 驗證工作階段

```
# Authenticate your Google Cloud account
gcloud auth login

# Set up Application Default Credentials (ADC) for SMT
gcloud auth application-default login

# Ensure your current project is set correctly
gcloud config set project $PROJECT_ID
```

(注意：系統提示時，請按照提供的網址授權帳戶，然後將驗證碼貼回終端機。)

2. 首先，請在新的 Cloud Shell 分頁中執行下列指令，找出第一個實體分片的外部 IP：

```
gcloud compute instances describe mysql-physical-1 --zone=$ZONE --
format='get(networkInterfaces[0].accessConfigs[0].natIP)'
```

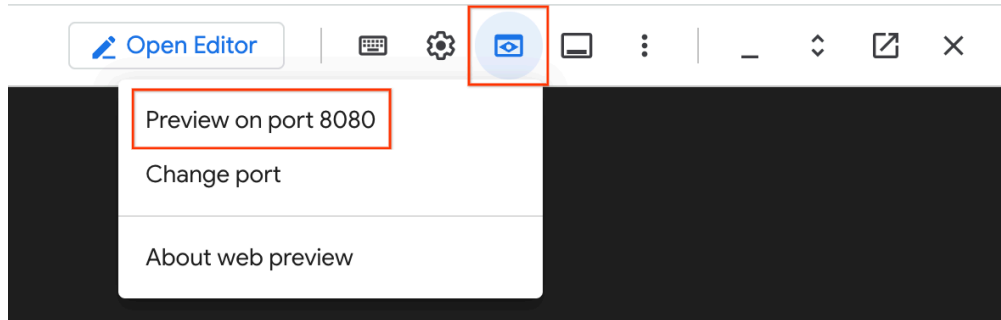
3. 列印目標 Spanner 執行個體詳細資料，以供設定 SMT 時使用。

```
echo "Project ID: $PROJECT_ID"
echo "Instance ID: $SPANNER_INSTANCE_NAME"
echo "Database Name: $SPANNER_DATABASE_NAME"
```

4. 啟動網頁版 UI：

```
gcloud alpha spanner migrate web --port=8080
```

5. 在 Cloud Shell 視窗的右上角，按一下「Web Preview」(網頁預覽) 圖示 (看起來像眼睛)，然後選取「Preview on port 8080」(在通訊埠 8080 上預覽)。系統會在新的瀏覽器分頁中開啟 SMT 使用者介面。



6. 在 SMT 網頁版 UI 中，選取「Connect to database」(連線至資料庫)。

7. 填寫連線詳細資料：

- 資料庫類型：MySQL
- 主機：(貼上步驟 2 中的 IP 位址)
- 通訊埠：3306
- 使用者： `datastream_user`
- 密碼： `complex_password_123`
- 資料庫名稱： `shard0_db`

如要產生基準結構定義，我們只需要將 SMT 連線至其中一個相同的邏輯分片。

8. 按一下右上角的編輯按鈕，設定 Spanner 資料庫。

9. 輸入目標 Spanner 詳細資料：

- 專案 ID：(貼上步驟 3 中的專案 ID)
- Spanner 執行個體：(貼上步驟 3 中的執行個體 ID)

10. 按一下「測試連線」。

11. 通過驗證後，按一下「連結」。SMT 會分析來源資料庫，並提供基準 Spanner 結構定義。

Spanner migration tool

Spanner database is not configured, click on edit button to configure.

Connect to Source Database

Database Engine*
MySQL

Sharded Migration
No

Preview

Connection Detail

Host name*
[REDACTED]

Port*
3306

User name*
datastream_user

Password*
[REDACTED]

Database Name*
shard0_db

Spanner Dialect
Spanner dialect*
Google Standard SQL Dialect

Test Connection

Connect

Cancel

3. 套用結構定義修改內容

現在我們要重塑結構定義，涵蓋複雜的遷移情境。

本節中套用的結構定義修改，旨在示範本程式碼研究室的簡單用途。在實際工作環境中進行遷移時，我們強烈建議您參閱 Cloud Spanner 的[結構定義設計最佳做法](#)，確保資料庫效能和擴充性達到最佳狀態。

在 SMT UI 的結構定義編輯器中，執行下列動作：

A. 重新命名 LegacyRegion 資料欄：

- 按一下左側導覽窗格中的 **Customers** 表格。系統預設會開啟「欄」分頁。
- 按一下「Spanner」部分中的「編輯」按鈕。
- 在 Spanner 結構定義檢視畫面中找出 **LegacyRegion** 資料欄。
- 在資料欄名稱對話方塊中輸入 **LoyaltyTier**，將 Spanner 資料欄名稱變更為 **LoyaltyTier**。
- 按一下「儲存並轉換」。

MYSQL SPANNER DRAFT

Customers SKIP TABLE

Filter Filter by table r SKIP RESTORE

COLUMNS PRIMARY KEY FOREIGN KEY INTERLEAVE CHECK CONSTRAINTS SQL

Status Object Name

shard0_db

Tables (2)

Customers

Orders

Sequences (0)

MYSQL

S No.	Name	Type	Auto-Generated	Max Length	Pk	Nullable
1	CustomerId	int	None	10		Not Null
2	CustomerName	varchar	None	255		
3	CreditLimit	decimal	None	10		Not Null
4	LegacyRegion	varchar	None	50		

Spanner

EDIT

ADD COLUMN

ADD SEQUENCE

Name	Type	Auto-Generated	Skip Range Min	Skip Range Max	Start Counter With	Max Length	Pk	Nullable
CustomerId	INT64	None						Not Null
CustomerName	STRING	None				255		
CreditLimit	NUMERIC	None						Not Null
LegacyRegion	STRING	None				50		

Customers

SKIP TABLE

16

COLUMNSPRIMARY KEYFOREIGN KEYINTERLEAVECHECK CONSTRAINTSSQL

MySQL

S No.	Name	Type	Auto-Generated	Max Length	Pk	Nullable
1	CustomerId	int	None	10	☒	Not Null
2	CustomerName	varchar	None	255		
3	CreditLimit	decimal	None	10		Not Null
4	LegacyRegion	varchar	None	50		

Spanner

Name	Type	Auto-Generated	Skip Range Min	Skip Range Max	Start Counter With	Max Length
CustomerId	INT64	None				
CustomerName	STRING	None				255
CreditLimit	NUMERIC	None				
LoyaltyTier	STRING	None				50

CANCEL

SAVE & CONVERT

B. 放寬檢查限制：

- 在 Customers 表格中，前往「Check Constraints」(檢查限制) 分頁。
- 找出 CHK_CreditLimit 限制條件。按一下「編輯」(鉛筆) 圖示。
- 將條件從 CreditLimit > 1000 變更為 CreditLimit > 0。(這會刻意導致信用額度較低的資料列無法反向遷移，並掉入 DLQ)。

Customers

SKIP TABLE

16

COLUMNSPRIMARY KEYFOREIGN KEYINTERLEAVECHECK CONSTRAINTSSQL

MySQL

S. No.	Constraint Name	Condition
1	CHK_CreditLimit	(`CreditLimit` > 1000)

Spanner

S. No.	Constraint Name	Condition
1	CHK_CreditLimit	(`CreditLimit` > 0)

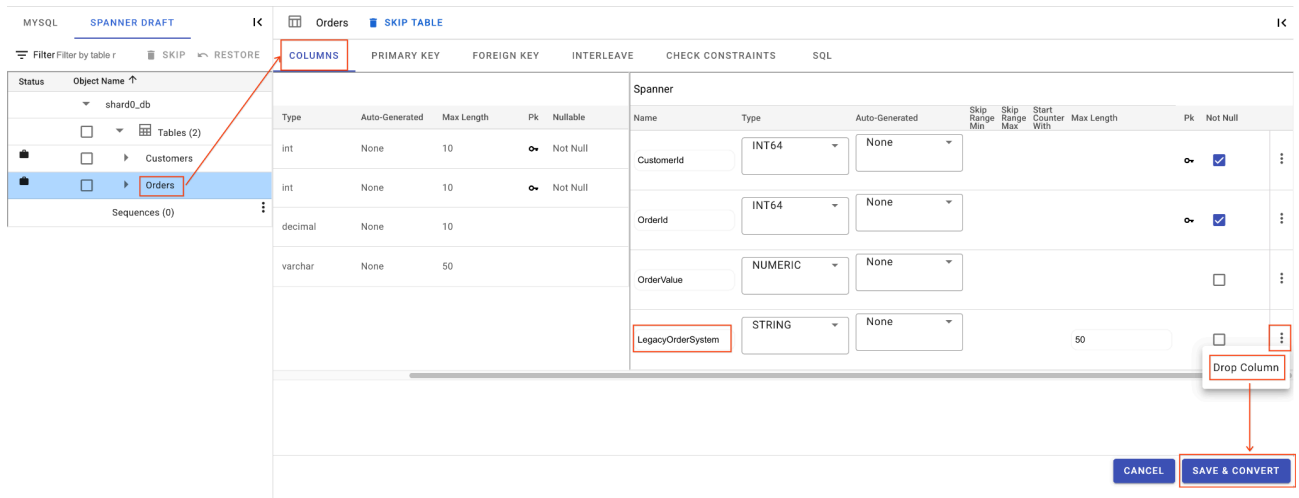
+ Add Constraint

CANCEL

SAVE & CONVERT

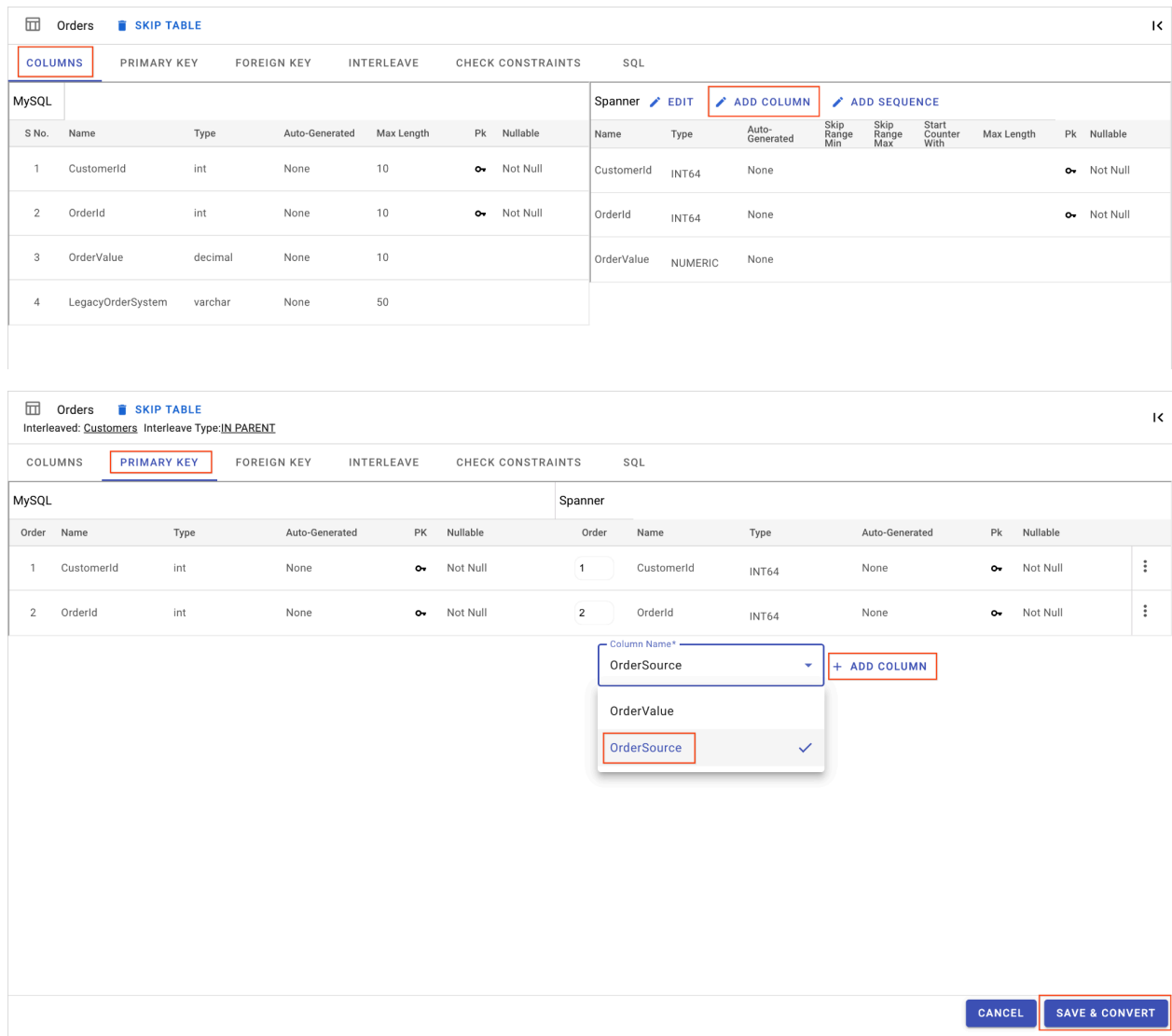
C. 捨棄 LegacyOrderSystem 資料欄：

- 按一下 Orders 表格，系統預設會開啟「欄」分頁。
- 按一下「Spanner」部分中的「編輯」按鈕。
- 在 Spanner 結構定義檢視畫面中找出 LegacyOrderSystem 資料欄。
- 按一下旁邊的三點選單圖示，然後選取「捨棄資料欄」。
- 按一下「儲存並轉換」。



D. 新增 OrderSource 資料欄並設為主鍵：

- 仍在 **Orders** 表格中，按一下「新增資料欄」。將其命名為 **OrderSource**，並將類型設為 **STRING**（長度為 **50**），不自動產生，並將 **IsNullable** 設為 **No**。
- 前往「主鍵」分頁。
- 按一下「編輯」，然後從「資料欄名稱」下拉式選單中選擇 **OrderSource**。
- 按一下「新增資料欄」，然後點選「儲存並轉換」。



E. 交錯顯示訂單表格：

- 在 `Orders` 表格中，找到主要表格檢視畫面中的「交錯」分頁。
- 將父項資料表設為 `Customers`。
- 選擇「交錯類型」 `IN PARENT` 和「刪除時的動作」 `NO ACTION`。
- 按一下 [儲存]。

The screenshot displays the Spanner migration tool interface. At the top, it shows the project ID and instance ID. Below this, the 'Configure Schema (Google Standard SQL Dialect)' section is active. The main workspace is divided into three panels. The left panel shows a tree view of the database schema with 'Orders' selected. The middle panel shows the configuration for the 'Orders' table, with the 'INTERLEAVE' tab selected. It shows 'Customers' as the table to interleave with, 'IN PARENT' as the interleave type, and 'NO ACTION' as the delete action. A 'SAVE' button is highlighted. The right panel shows 'ISSUES AND SUGGESTIONS' with a warning about storage consumption for the 'Orders' table.

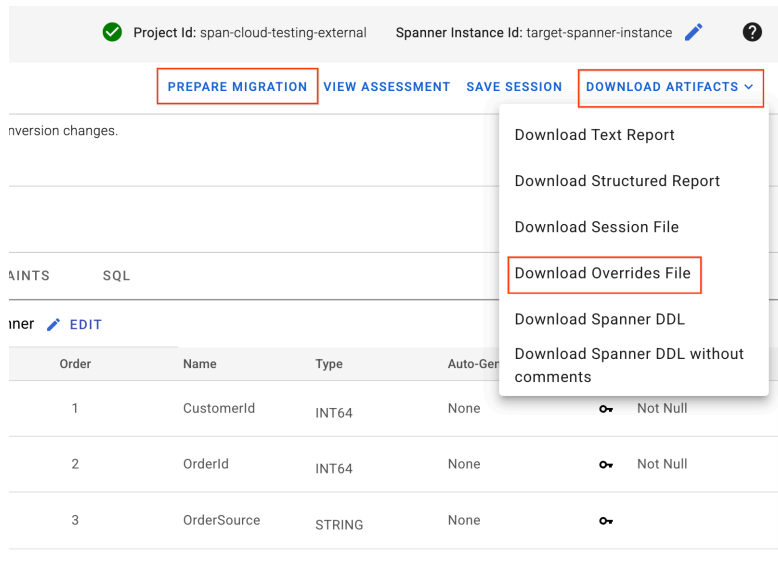
- SMT UI 支援多種結構定義修改作業。如需完整清單，請參閱[官方說明文件](#)。
- SMT 中的「問題和建議」面板 (位於主要工作區右側) 會標示發生的任何結構定義轉換變更。詳情請參閱[本文](#)。

4. 下載覆寫檔案並套用結構定義

- 在 SMT 使用者介面的右上角，找到「Download Artifacts」按鈕。選取「下載覆寫檔案」選項。將這個檔案儲存到本機電腦。這個檔案包含我們剛才所做的所有結構定義對應變更，將由 **Dataflow** 管道使用。

Spanner 遷移工具執行時會產生多個檔案。每個檔案都有自己的功能，從儲存現有的遷移狀態，到為分析的結構定義產生報表。如要進一步瞭解這些「構件」，請參閱[這篇文章](#)。

- 按一下「準備遷移」。



3. 從下拉式選單中選擇「遷移模式」做為 `Schema` 。

4. 輸入目標 Spanner 資料庫：`sharded-target-db`

Select Source > Configure Schema (Google Standard SQL Dialect) > **Prepare Migration**

Source and Target Database definitions

Title	Source	Destination
Database Type	mysql	Spanner
Number of tables	2	2
Number of indexes	0	0

Migration Mode:
Schema

Prerequisites

Before we begin, please ensure you have done the following:

Target details:

1. Configure Spanner Database **Configure**

Migrate

5. 按一下「遷移」。

6. SMT 會套用 DDL 並建立 Spanner 資料庫。SMT 程序完成後，即可放心在 Cloud Shell 中停止程序 (Ctrl+C)。

5. 在 Cloud Spanner 中驗證結構定義

確認資料表已在 Spanner 資料庫中建立。

```
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \  
  --instance=$SPANNER_INSTANCE_NAME \  
  --sql="SELECT table_name FROM information_schema.tables WHERE table_schema = '' ORDER BY  
table_name"
```

您應該會看到以下的輸出內容：

```
table_name: Customers
table_name: Orders
```

選用：如要檢查實際的 Spanner DDL，確認檢查限制、交錯和額外資料欄已套用，請執行下列指令：

```
gcloud spanner databases ddl describe $SPANNER_DATABASE_NAME \
--instance=$SPANNER_INSTANCE_NAME
```

預期輸出內容：

```
CREATE TABLE Customers (
  CustomerId INT64 NOT NULL,
  CustomerName STRING(255),
  CreditLimit NUMERIC NOT NULL,
  LoyaltyTier STRING(50),
  CONSTRAINT CHK_CreditLimit CHECK(`CreditLimit` > 0),
) PRIMARY KEY(CustomerId);

CREATE TABLE Orders (
  CustomerId INT64 NOT NULL,
  OrderId INT64 NOT NULL,
  OrderValue NUMERIC,
  OrderSource STRING(50) NOT NULL,
) PRIMARY KEY(CustomerId, OrderId, OrderSource),
INTERLEAVE IN PARENT Customers ON DELETE NO ACTION;
```

7. 初始化變更資料擷取 (CDC)

在本節中，您將設定遷移作業的「記錄器」。在大量載入資料前設定 Datastream 和 Pub/Sub，可確保系統擷取並將來源資料庫的所有變更排入佇列，避免遷移期間發生資料遺失。這是即時遷移的必要設定。

由於我們的架構涉及兩部實體伺服器，因此必須建立兩個不同的 Datastream 來源設定檔和兩個 Datastream 串流。這兩個串流都會寫入單一 Google Cloud Storage (GCS) bucket，做為 Dataflow 管道的統一來源。

1. 建立 Cloud Storage bucket

Datastream 需要目的地來儲存擷取的變更事件。現在來建立 GCS bucket。

```
export BUCKET_NAME="migration-${PROJECT_ID}-bucket"
gcloud storage buckets create gs://${BUCKET_NAME} --location=$REGION
```

2. 建立 Datastream 連線設定檔

我們需要兩個不同的 MySQL 來源連線設定檔 (每個實體分片各一個)，以及一個 Cloud Storage 的目標連線設定檔。

取得來源 IP 位址

首先，擷取兩個 Compute Engine VM 的外部 IP 位址，並儲存為環境變數：

```
export MYSQL_IP_1=$(gcloud compute instances describe mysql-physical-1 --zone=$ZONE --
format='get(networkInterfaces[0].accessConfigs[0].natIP)')

export MYSQL_IP_2=$(gcloud compute instances describe mysql-physical-2 --zone=$ZONE --
format='get(networkInterfaces[0].accessConfigs[0].natIP)')
```

建立來源連線設定檔 (Compute Engine 上的 MySQL)

使用先前建立的 `datastream_user` 建立 Datastream 連線設定檔。

```
# Create Source Profile for Physical Shard 1
export SQL_CP_NAME_1="mysql-src-cp-1"
gcloud datastream connection-profiles create $SQL_CP_NAME_1 \
  --location=$REGION \
  --type=mysql \
  --mysql-hostname=$MYSQL_IP_1 \
  --mysql-port=3306 \
  --mysql-username=datastream_user \
  --mysql-password=complex_password_123 \
  --display-name="MySQL Source 1 (Physical Shard 1)"

# Create Source Profile for Physical Shard 2
export SQL_CP_NAME_2="mysql-src-cp-2"
gcloud datastream connection-profiles create $SQL_CP_NAME_2 \
  --location=$REGION \
  --type=mysql \
  --mysql-hostname=$MYSQL_IP_2 \
  --mysql-port=3306 \
  --mysql-username=datastream_user \
  --mysql-password=complex_password_123 \
  --display-name="MySQL Source 2 (Physical Shard 2)"
```

注意：Datastream 會透過這些 VM 的公開 IP 連線，這是允許的行為，因為我們稍早已在防火牆規則中新增 0.0.0.0/0。在正式環境中，您會嚴格允許 Datastream 的特定公用 IP 範圍。

建立目的地連線設定檔 (Cloud Storage)：

這會指向新建立 bucket 的根目錄。

```
export GCS_CP_NAME="gcs-dest-cp"
gcloud datastream connection-profiles create $GCS_CP_NAME \
  --location=$REGION \
  --type=google-cloud-storage \
  --bucket=$BUCKET_NAME \
  --root-path=/ \
  --display-name="GCS Destination" --force
```

3. 建立 Datastream 串流

現在要建立兩個 CDC 串流。串流 1 會擷取 `shard0_db` 和 `shard1_db`。串流 2 會擷取 `shard2_db` 和 `shard3_db`。兩個串流都會以 Avro 格式寫入同一個 GCS 值區。

```
# Stream for Physical Shard 1
export STREAM_NAME_1="mysql-to-spanner-stream-1"
export GCS_STREAM_PATH_1="data/${STREAM_NAME_1}"

gcloud datastream streams create $STREAM_NAME_1 \
  --location=$REGION \
  --display-name="MySQL Source 1 CDC Stream" \
  --source=$SQL_CP_NAME_1 \
  --destination=$GCS_CP_NAME \
  --mysql-source-config=<(echo "includeObjects:
mysqlDatabases:
- database: 'shard0_db'
- database: 'shard1_db'" ) \
  --gcs-destination-config=<(echo "path: ${GCS_STREAM_PATH_1}/
fileRotationMb: 5
fileRotationInterval: 15s
avroFileFormat: {}") \
  --backfill-none

# Stream for Physical Shard 2
export STREAM_NAME_2="mysql-to-spanner-stream-2"
export GCS_STREAM_PATH_2="data/${STREAM_NAME_2}"

gcloud datastream streams create $STREAM_NAME_2 \
  --location=$REGION \
  --display-name="MySQL Source 2 CDC Stream" \
  --source=$SQL_CP_NAME_2 \
  --destination=$GCS_CP_NAME \
  --mysql-source-config=<(echo "includeObjects:
mysqlDatabases:
- database: 'shard2_db'
- database: 'shard3_db'" ) \
  --gcs-destination-config=<(echo "path: ${GCS_STREAM_PATH_2}/
fileRotationMb: 5
fileRotationInterval: 15s
avroFileFormat: {}") \
  --backfill-none
```

使用較小的檔案輪替設定 (5 MB 或 15 秒)，有助於在程式碼研究室中更快看到複製的變更。

這個指令可能需要一段時間才能完成。查看狀態：`gcloud datastream streams describe $STREAM_NAME_1 --location=$REGION`。

4. 啟動 Datastream 串流

啟動這兩個串流，開始記錄變更。

```
gcloud datastream streams update $STREAM_NAME_1 \  
  --location=$REGION \  
  --state=RUNNING  
  
gcloud datastream streams update $STREAM_NAME_2 \  
  --location=$REGION \  
  --state=RUNNING
```

檢查狀態：您可以執行 `gcloud datastream streams describe $STREAM_NAME_1 --location=$REGION`。狀態一開始會是 `STARTING`，稍後會變更為 `RUNNING`。請等待兩者都完全執行後，再開始即時遷移。

5. 為 GCS 通知設定 Pub/Sub

當任一 Datastream 串流將新檔案寫入 GCS bucket 時，Dataflow 都需要立即收到通知。我們將設定 GCS，將通知傳送至單一 Pub/Sub 主題。

建立 Pub/Sub 主題：

```
export PUBSUB_TOPIC="datastream-gcs-updates"  
gcloud pubsub topics create $PUBSUB_TOPIC
```

建立 GCS 通知

在 `data/` 前置字元下建立任何物件時，通知主題 (涵蓋兩個串流)。

```
gcloud storage buckets notifications create gs://${BUCKET_NAME} --  
topic=projects/$PROJECT_ID/topics/$PUBSUB_TOPIC --payload-format=json --object-prefix=data/
```

建立 Pub/Sub 訂閱項目

建立訂閱項目，並為 Dataflow 設定建議的確認期限。

```
export PUBSUB_SUBSCRIPTION="datastream-gcs-sub"  
gcloud pubsub subscriptions create $PUBSUB_SUBSCRIPTION \  
  --topic=$PUBSUB_TOPIC \  
  --ack-deadline=600
```

8. 自訂轉換

由於我們的 Spanner 結構定義與 MySQL 結構定義不同 (因為我們透過 SMT 網頁介面新增及捨棄了資料欄)，因此開箱即用的 Dataflow 遷移作業會失敗。Dataflow 需要相關指令，才能在正向 (MySQL 至 Spanner) 和反向 (Spanner 至 MySQL) 管道中對應這些差異。

此外，由於我們正在執行分片反向遷移作業，Dataflow 需要路由機制，才能在反向複製期間，瞭解更新後的 Spanner 資料列屬於哪個邏輯分片 (`shard0_db` 、 `shard1_db` 等)。

我們會使用 Google 提供的 Spanner 自訂分片範本，編寫自訂轉換 JAR 檔案，藉此達成目標。

1. 下載自訂分片範本

在 Cloud Shell 中，下載 Google Cloud Dataflow 範本存放區，然後前往自訂分片資料夾：

```
git clone https://github.com/GoogleCloudPlatform/DataflowTemplates.git
cd DataflowTemplates/v2/spanner-custom-shard
```

2. 設定資料轉換邏輯

我們需要編輯 `CustomTransformationFetcher.java` 檔案。

- **向前移轉 (`toSpannerRow`)**：使用 MySQL 中的 `LegacyOrderSystem` 欄位填入新加入的 `OrderSource` 欄位。
- **反向遷移 (`toSourceRow`)**：重新填入 MySQL 要求的已捨棄 `LegacyOrderSystem` 欄，並從 Spanner 的 `OrderSource` 衍生該欄。

編輯 `CustomTransformationFetcher.java` 檔案。不必手動開啟文字編輯器，執行下列指令即可自動以自訂邏輯覆寫範本檔案：

```

cat << 'EOF' > src/main/java/com/custom/CustomTransformationFetcher.java
package com.custom;

import com.google.cloud.teleport.v2.spanner.exceptions.InvalidTransformationException;
import com.google.cloud.teleport.v2.spanner.utils.ISpannerMigrationTransformer;
import com.google.cloud.teleport.v2.spanner.utils.MigrationTransformationRequest;
import com.google.cloud.teleport.v2.spanner.utils.MigrationTransformationResponse;
import java.util.HashMap;
import java.util.Map;

public class CustomTransformationFetcher implements ISpannerMigrationTransformer {

    @Override
    public void init(String customParameters) {}

    @Override
    public MigrationTransformationResponse toSpannerRow(MigrationTransformationRequest request)
        throws InvalidTransformationException {
        if (request.getTableName().equals("Orders")) {
            Map<String, Object> requestRow = request.getRequestRow();
            Map<String, Object> responseRow = new HashMap<>();

            Object legacySysObj = requestRow.get("LegacyOrderSystem");
            String legacySys = (legacySysObj != null) ? (String) legacySysObj : "UNKNOWN_SYSTEM";

            // Transform: Trim the string to remove everything after the first underscore
            String orderSource = legacySys;
            if (legacySys.contains("_")) {
                orderSource = legacySys.substring(0, legacySys.indexOf('_'));
            }

            // Populate the new Spanner column (e.g., "WebStore_v1" becomes "WebStore")
            responseRow.put("OrderSource", orderSource);

            return new MigrationTransformationResponse(responseRow, false);
        }

        return new MigrationTransformationResponse(new HashMap<>(), false);
    }

    @Override
    public MigrationTransformationResponse toSourceRow(MigrationTransformationRequest request)
        throws InvalidTransformationException {
        if (request.getTableName().equals("Orders")) {
            Map<String, Object> requestRow = request.getRequestRow();
            Map<String, Object> responseRow = new HashMap<>();

            // Safely fetch the Spanner OrderSource
            Object sourceObj = requestRow.get("OrderSource");
            String source = (sourceObj != null) ? (String) sourceObj : "UNKNOWN_SYSTEM";
            String legacySys = "'" + source + "_v1'";

```

```

// Transform: Append a suffix to visibly prove the reverse transformation worked
// e.g., "WebStore" becomes "WebStore_v1"
responseRow.put("LegacyOrderSystem", legacySys);

return new MigrationTransformationResponse(responseRow, false);
}

return new MigrationTransformationResponse(new HashMap<>(), false);
}

@Override
public MigrationTransformationResponse transformFailedSpannerMutation(
    MigrationTransformationRequest request) throws InvalidTransformationException {
    return new MigrationTransformationResponse(new HashMap<>(), false);
}
}
EOF

```

自訂轉換主要用於轉換現有資料欄的值，或填入新資料欄。系統會隱含處理重新命名或捨棄的資料欄邏輯，**不需要自訂轉換**。因此，在上述函式中，我們不會明確處理資料表 `Customers` 中重新命名的 `LegacyRegion` 資料欄。

3. 設定反向分片邏輯

Dataflow 會在反向複製期間使用 `CustomShardIdFetcher.java`，判斷 Spanner 變異的路由位置。我們會使用 `CustomerId` 主鍵和模數 (%4) 邏輯，動態將記錄轉送回正確的邏輯分片。

使用 `cat` 編輯 `CustomShardIdFetcher.java` 檔案，然後將內容完全替換為下列程式碼：

```

cat << 'EOF' > src/main/java/com/custom/CustomShardIdFetcher.java
package com.custom;

import com.google.cloud.teleport.v2.spanner.utils.IShardIdFetcher;
import com.google.cloud.teleport.v2.spanner.utils.ShardIdRequest;
import com.google.cloud.teleport.v2.spanner.utils.ShardIdResponse;
import java.util.Map;

public class CustomShardIdFetcher implements IShardIdFetcher {

    @Override
    public void init(String parameters) {}

    @Override
    public ShardIdResponse getShardId(ShardIdRequest shardIdRequest) {
        Map<String, Object> keys = shardIdRequest.getSpannerRecord
();

        // Use the Primary Key to identify the correct logical shard
        if (keys != null && keys.containsKey("CustomerId")) {
            long customerId = Long.parseLong(keys.get("CustomerId").toString());
            long shardIdx = customerId % 4;

            ShardIdResponse response = new ShardIdResponse();
            response.setLogicalShardId("shard" + shardIdx + "_db");
            return response;
        }

        return new ShardIdResponse();
    }
}
EOF

```

4. 建構及上傳 JAR

自訂 Java 邏輯編寫完成後，我們需要將其編譯為 JAR 檔案，並上傳至先前建立的 Google Cloud Storage bucket，供 Dataflow 存取。

在 Cloud Shell 中執行下列指令：

```
# Return to DataflowTemplates directory
cd ../../

# Build the JAR using Maven
mvn clean install -DskipTests -Dcheckstyle.skip -Dspotless.check.skip=true -Djib.skip -pl
v2/spanner-custom-shard -am

# Upload the JAR to GCS
export CUSTOM_JAR_PATH="gs://{BUCKET_NAME}/custom-logic/spanner-custom-shard-1.0.jar"

gcloud storage cp v2/spanner-custom-shard/target/spanner-custom-shard-1.0-SNAPSHOT.jar
$CUSTOM_JAR_PATH

# Return to home directory
cd ~
```

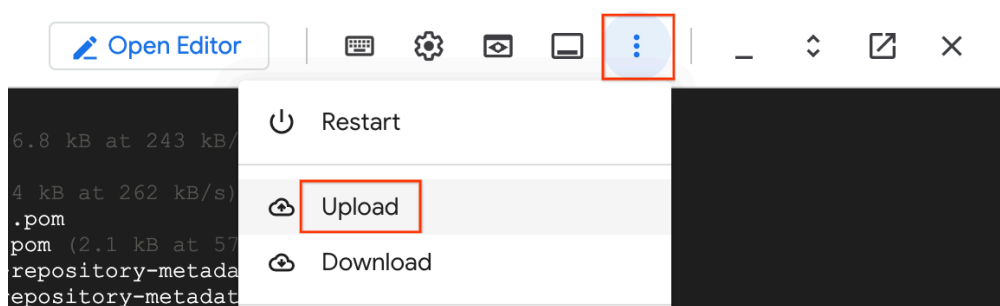
9. 從 MySQL 大量遷移資料至 Spanner

有了 Spanner 結構定義並建構自訂轉換 JAR 後，我們現在可以將現有資料從 MySQL 資料庫複製到 Cloud Spanner。您將使用 `sourcedb to spanner` Dataflow 彈性範本，這個範本專為從 JDBC 可存取的資料庫大量複製資料到 Spanner 而設計。

1. 上傳結構定義覆寫檔案

在第 6 節中，您使用 SMT 網頁版 UI 下載了 Spanner Overrides JSON 檔案。我們需要將這個檔案上傳至 GCS bucket，Dataflow 才能使用該檔案對應結構定義差異 (例如重新命名的資料欄)。

1. 在 Cloud Shell 中，按一下三點選單 (「更多」)，然後選取「上傳」。



2. 選取您先前下載的覆寫 JSON 檔案 (例如 `spanner_overrides.json`)。
3. 將其移至 GCS bucket：

```
export OVERRIDES_FILE="spanner_overrides.json" # Change this if your downloaded file has a
different name

export GCS_OVERRIDES_PATH="gs://${BUCKET_NAME}/config/${OVERRIDES_FILE}"

gcloud storage cp ~/${OVERRIDES_FILE} $GCS_OVERRIDES_PATH
```

2. 建立及上傳分片設定檔

Dataflow 必須知道如何連線至兩個實體 VM 中的所有四個邏輯分片。我們會為此建立 `sharding.json` 檔案。

在 Cloud Shell 中執行下列指令，產生並上傳設定：

```

cat <<EOF > sharding.json
{
  "configType": "dataflow",
  "shardConfigurationBulk": {
    "schemaSource": {
      "dataShardId": "mysql-physical-1",
      "host": "${MYSQL_IP_1}",
      "user": "datastream_user",
      "password": "complex_password_123",
      "port": "3306",
      "dbName": "shard0_db"
    },
    "dataShards": [
      {
        "dataShardId": "mysql-physical-1",
        "host": "${MYSQL_IP_1}",
        "user": "datastream_user",
        "password": "complex_password_123",
        "port": "3306",
        "dbName": "",
        "namespace": "namespace-mysql-1",
        "databases": [
          {
            "dbName": "shard0_db",
            "databaseId": "shard0_db",
            "refDataShardId": "mysql-physical-1"
          },
          {
            "dbName": "shard1_db",
            "databaseId": "shard1_db",
            "refDataShardId": "mysql-physical-1"
          }
        ]
      },
      {
        "dataShardId": "mysql-physical-2",
        "host": "${MYSQL_IP_2}",
        "user": "datastream_user",
        "password": "complex_password_123",
        "port": "3306",
        "dbName": "",
        "namespace": "namespace-mysql-2",
        "databases": [
          {
            "dbName": "shard2_db",
            "databaseId": "shard2_db",
            "refDataShardId": "mysql-physical-2"
          },
          {
            "dbName": "shard3_db",
            "databaseId": "shard3_db",
            "refDataShardId": "mysql-physical-2"
          }
        ]
      }
    ]
  }
}

```

```

    }
  ]
}
]
}
}
EOF

export GCS_SHARDING_PATH="gs://${BUCKET_NAME}/config/sharding.json"
gcloud storage cp sharding.json $GCS_SHARDING_PATH

```

3. 執行大量遷移 Dataflow 工作

我們將使用 **Sourcedb to Spanner** Flex 範本。由於這是採用自訂轉換的分片遷移作業，因此我們會傳遞覆寫檔案、分片設定和自訂 Java JAR。

```

export JOB_NAME="mysql-sharded-bulk-to-spanner-$(date +%Y%m%d-%H%M%S)"
export OUTPUT_DIR="gs://${BUCKET_NAME}/bulk-migration"

gcloud dataflow flex-template run $JOB_NAME \
  --project=$PROJECT_ID \
  --region=$REGION \
  --template-file-gcs-location="gs://dataflow-
templates-${REGION}/latest/flex/Sourcedb_to_Spanner_Flex" \
  --max-workers=2 \
  --num-workers=1 \
  --worker-machine-type=n2-highmem-8 \
  --parameters \
sourceConfigURL=$GCS_SHARDING_PATH,\
instanceId=$SPANNER_INSTANCE_NAME,\
databaseId=$SPANNER_DATABASE_NAME,\
projectId=$PROJECT_ID,\
outputDirectory=$OUTPUT_DIR,\
username=datastream_user,\
password=complex_password_123,\
schemaOverridesFilePath=$GCS_OVERRIDES_PATH,\
transformationJarPath=$CUSTOM_JAR_PATH,\
transformationClassName=com.custom.CustomTransformationFetcher

```

重要參數說明：

- `sourceConfigURL`：我們建立的 `sharding.json` 檔案路徑。這會告知 Dataflow 如何連線至兩個實體 VM 中的四個邏輯 MySQL 分片。
- `schemaOverridesFilePath`：從 SMT 網頁介面下載的 JSON 檔案路徑。這會指示 Dataflow 如何處理我們所做的結構定義修改 (例如捨棄 `LegacyRegion` 欄位和縮緊檢查限制)。
- `transformationJarPath`：我們在上一節中建構的已編譯 Java JAR 檔案 GCS 路徑。其中包含要執行自訂轉換的實際程式碼。
- `transformationClassName`：JAR 內 Java 類別的完整名稱，該類別會實作轉送遷移邏輯 (`com.custom.CustomTransformationFetcher`)。

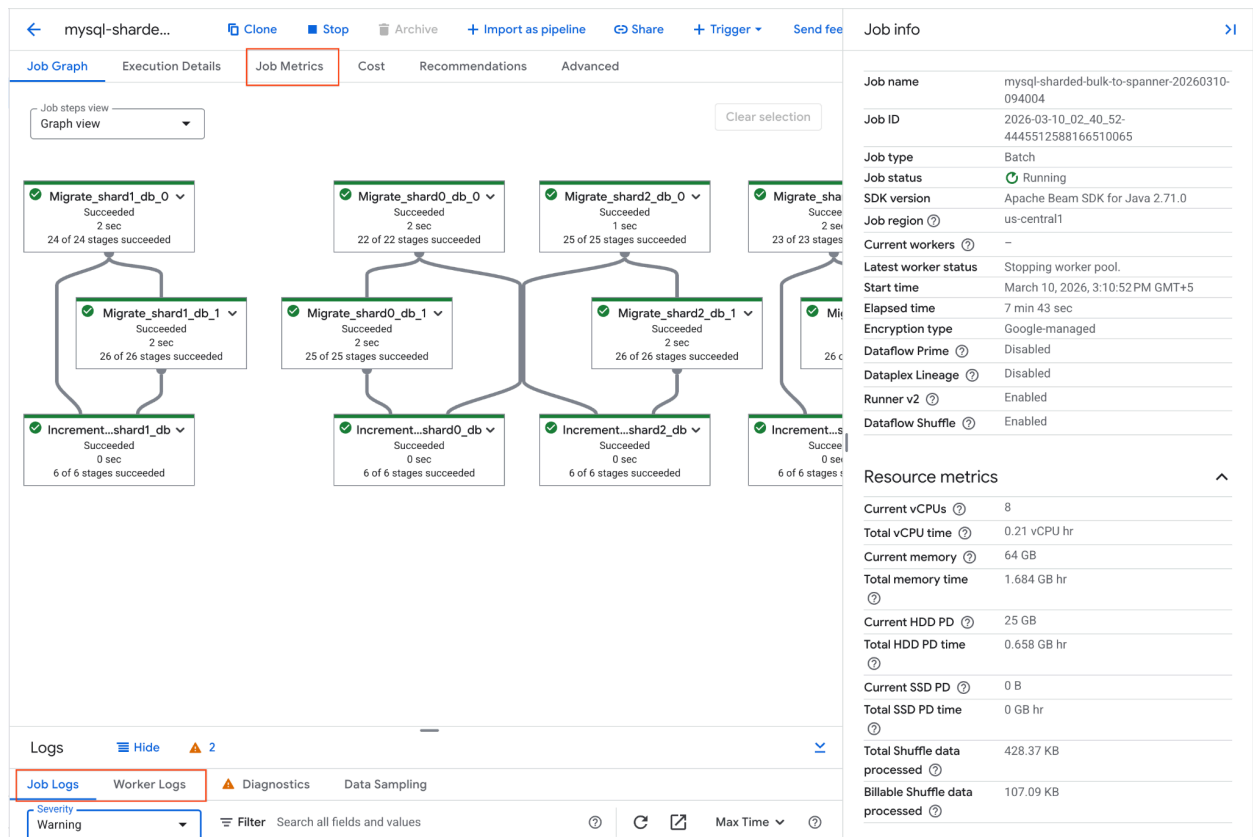
- `outputDirectory`：Dataflow 寫入暫存檔案的位置，最重要的是無效信件佇列 (DLQ) 檔案。
- `maxWorkers`、`numWorkers`：控制 Dataflow 工作的縮放。這個小型資料集會維持低值。
- `instanceId`、`databaseId`、`projectId`：指定目標 Cloud Spanner 執行個體和資料庫。

網路注意事項：這項作業會透過公開 IP 連線至 Cloud SQL 執行個體。這是因為您先前已將 `0.0.0.0/0` 新增至執行個體的已授權網路。這樣一來，具有外部 IP 的 Dataflow 工作人員 VM 就能連線至資料庫。

4. 監控 Dataflow 工作

您可以在 Google Cloud 控制台中追蹤工作進度：

1. 前往 Dataflow 的「Jobs」(工作) 頁面：[前往 Dataflow 的「Jobs」\(工作\) 頁面](#)
2. 找出名為 `mysql-sharded-bulk-to-spanner-...` 的工作，然後按一下。
3. 觀察工作圖表和指標。等待工作狀態變更為「Succeeded」。這項作業約需 5 至 15 分鐘。



- 如果工作發生問題，請查看 Dataflow 工作詳細資料頁面中的「記錄」分頁，瞭解錯誤訊息。
- 「工作指標」會提供工作進度和資源消耗量 (例如輸送量和 CPU 使用率) 的詳細資訊。

如需更詳盡的監控指南，請參閱[這篇文章](#)。

5. 驗證 Cloud Spanner 中的資料，並檢查無效信件佇列 (DLQ)

Dataflow 工作順利完成後，我們需要確認資料是否安全抵達，並檢查我們刻意設計為失敗的記錄。

什麼是 **DLQ**？無效信件佇列 (DLQ) 是儲存目錄，Dataflow 會將無法寫入 Spanner 的記錄安全地轉送至該目錄。Dataflow 不會讓整個遷移管道當機，而是將錯誤的資料列放入 DLQ (以 `outputDirectory` 中的 JSON 檔案形式)，方便您稍後檢查、修正並重試。

A. 驗證遷移資料的整體健康狀態：

使用 `gcloud` CLI 對合併的 Spanner 資料庫執行幾項快速健康檢查，確保有效記錄已正確遷移，且自訂 JAR 已填入額外資料欄。

```
# 1. Verify total Customer count
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="SELECT COUNT(*) as TotalCustomers FROM Customers"

# 2. Verify total Orders count (Total minus the orphan record)
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="SELECT COUNT(*) as TotalOrders FROM Orders"

# 3. Verify the Custom Transformation on OrderSource worked
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="SELECT CustomerId, OrderId, OrderSource FROM Orders LIMIT 3"

# 4. Verify that renamed column LoyaltyTier has the correct data
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="SELECT CustomerId, CustomerName, LoyaltyTier FROM Customers LIMIT 3"
```

預期輸出內容：

```
TotalCustomers: 16
TotalOrders: 19

CustomerId: 1
OrderId: 201
OrderSource: MobileApp

CustomerId: 2
OrderId: 301
OrderSource: CallCenter

CustomerId: 3
OrderId: 401
OrderSource: InStore

CustomerId: 1
CustomerName: Agnes N.
LoyaltyTier: NORTHEAST

CustomerId: 2
CustomerName: Brian K.
LoyaltyTier: SOUTHWEST

CustomerId: 3
CustomerName: Cathy Z.
LoyaltyTier: CENTRAL
```

- 已成功遷移 Customers 資料表中的所有資料列。
- 我們發現 Orders 資料表中有 **1 列失敗**，這是因為 Spanner 中的 `INTERLEAVE IN PARENT` 是孤立的子項，Customers 資料表中沒有對應的資料列。CustomerId 99

B. 檢查 DLQ 中的刻意失敗：

大量遷移管道建立的無效信件佇列 (DLQ) 資料夾中，會記錄上述失敗情形。

1. 前往 Google Cloud 控制台的「Cloud Storage」。
2. 前往 bucket 並開啟 `bulk-migration/dlq/severe` 資料夾。
3. 檢查其中的 JSON 檔案。您會看到 Orders 列，其中包含孤立的 CustomerId。
4. 如要重試大量遷移 DLQ 錯誤，請按照[這篇文章](#)中的步驟操作。

從 Cloud SQL 大量載入資料至 Cloud Spanner 的作業已完成。下一步是設定即時複製功能，擷取持續變更。

10. 開始即時遷移 (CDC)

大量資料載入作業完成後，您將啟動持續執行的 Dataflow 串流工作。這項工作會讀取 Datastream 寫入 GCS bucket 的變更資料擷取 (CDC) 事件，並近乎即時地將這些變更套用至 Cloud Spanner。

我們也會注入有效和刻意無效的資料，測試這個管道，觀察 Dataflow 如何處理即時複製作業，以及如何將失敗的作業轉送至無效信件佇列 (DLQ)。

1. 建立即時遷移分片設定檔

與大量遷移 (使用 JDBC 連線字串) 不同，即時遷移管道會從 GCS 讀取 Datastream 事件。這需要完全不同的 JSON 設定，將 Datastream 資料串流名稱和資料庫對應至邏輯 Spanner 分片。

在 Cloud Shell 中執行下列指令，建立並上傳即時分片設定：

```
cat <<EOF > live-sharding.json
{
  "StreamToDbAndShardMap": {
    "${STREAM_NAME_1}": {
      "shard0_db": "shard0_db",
      "shard1_db": "shard1_db"
    },
    "${STREAM_NAME_2}": {
      "shard2_db": "shard2_db",
      "shard3_db": "shard3_db"
    }
  }
}
EOF

export GCS_LIVE_SHARDING_PATH="gs://${BUCKET_NAME}/config/live-sharding.json"
gcloud storage cp live-sharding.json $GCS_LIVE_SHARDING_PATH
```

2. 執行 Live Migration Dataflow 工作

啟動串流 Dataflow 工作，從 GCS 讀取資料並寫入 Spanner。這個範本會使用 GCS Pub/Sub 通知，即時處理新檔案。

```
export JOB_NAME_CDC="mysql-sharded-cdc-to-spanner-$(date +%Y%m%d-%H%M%S)"
export DLQ_DIR_CDC="gs://${BUCKET_NAME}/live-migration"

gcloud dataflow flex-template run $JOB_NAME_CDC \
  --project=$PROJECT_ID \
  --region=$REGION \
  --worker-machine-type=n2-highmem-8 \
  --template-file-gcs-location="gs://dataflow-templates-${REGION}/latest/flex/Cloud_Datastream_to_Spanner" \
  --parameters \
  instanceId="$SPANNER_INSTANCE_NAME",\
  databaseId="$SPANNER_DATABASE_NAME",\
  projectId="$PROJECT_ID",\
  inputFileFormat="avro",\
  gcsPubSubSubscription="projects/${PROJECT_ID}/subscriptions/${PUBSUB_SUBSCRIPTION}",\
  shardingContextFilePath=$GCS_LIVE_SHARDING_PATH,\
  deadLetterQueueDirectory="$DLQ_DIR_CDC",\
  transformationJarPath=$CUSTOM_JAR_PATH,\
  transformationClassName="com.custom.CustomTransformationFetcher",\
  schemaOverridesFilePath=$GCS_OVERRIDES_PATH,\
  datastreamSourceType="mysql",\
  dlqRetryMinutes=1,\
  dlqMaxRetryCount=2
```

重要參數

- `gcsPubSubSubscription`：Pub/Sub 訂閱項目，用於監聽 GCS 的新檔案通知。這樣一來，工作就能在 Datastream 寫入變更時立即處理。
- `inputFileFormat="avro"`：告知 Dataflow 預期會收到 Datastream 傳送的 Avro 檔案。這必須與 Datastream 的「目的地」設定相符 (例如 `avroFileFormat` 與 `jsonFileFormat`)。
- `shardingContextFilePath`：JSON 檔案，將 Datastream 資料流對應至邏輯分片。
- `dlqRetryMinutes`：無效信件佇列重試之間的時間隔分鐘數。預設為 10。
- `dlqMaxRetryCount`：透過 DLQ 重試暫時性錯誤的次數上限。預設為 500。

在 [Dataflow Jobs 控制台](#) 中監控工作啟動情形。

3. 注入即時資料並觸發有意發生的失敗

Dataflow 串流工作啟動時 (可能需要 3 到 5 分鐘)，請透過 SSH 登入第一個實體 MySQL VM，並插入一些新記錄。我們會插入一筆有效記錄和一筆無效記錄。

透過 SSH 連入第一個實體分片：

```
gcloud compute ssh mysql-physical-1 --zone=$ZONE
```

登入 MySQL：

```
sudo mysql
```

在 `shard1_db` 上執行下列插入作業：

```
USE shard1_db;

-- 1. Valid Insert: 'MobileApp_v2' will be trimmed to 'MobileApp'
INSERT INTO Orders (CustomerId, OrderId, OrderValue, LegacyOrderSystem)
VALUES (4, 501, 99.99, 'MobileApp_v2');

-- 2. Invalid Insert (DLQ Test): This violates Interleave constraint as CustomerId 99999
doesn't exist in Customers table.
INSERT INTO Orders (CustomerId, OrderId, OrderValue, LegacyOrderSystem)
VALUES (99999, 502, 50.00, 'WebStore_v1');

-- 3. Valid Update
UPDATE Orders SET OrderValue = '1500' WHERE CustomerId = 5 AND OrderId = 202;

-- 4. Valid Delete
DELETE FROM Orders WHERE CustomerId = 5 AND OrderId = 203;

EXIT;
```

再次輸入 `exit`，返回 Cloud Shell 提示。

與大量遷移不同，這項工作是串流工作。由於這項作業會持續監聽新變更，因此狀態會維持 `Running`，直到手動取消為止。執行完畢後，即可立即繼續下一個步驟。

4. 驗證即時遷移資料並檢查 CDC DLQ

現在我們已插入資料，Datastream 會擷取 CDC 事件，而 Dataflow 會嘗試將這些事件套用至 Spanner。

A. 驗證 Spanner 中的有效 DML 變更

執行下列查詢，確認 `INSERT`、`UPDATE` 和 `DELETE` 事件已順利傳送至 Spanner，且自訂轉換已在插入和更新作業中觸發。

```
# 1. Verify INSERT: Should return the new row with transformed OrderSource
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="SELECT CustomerId, OrderId, OrderValue, OrderSource FROM Orders WHERE CustomerId =
4 AND OrderId = 501"

# 2. Verify UPDATE: Should show OrderValue changed to 1500
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="SELECT CustomerId, OrderId, OrderValue, OrderSource FROM Orders WHERE CustomerId =
5 AND OrderId = 202"

# 3. Verify DELETE: Should return 0, confirming the order was deleted
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="SELECT COUNT(*) FROM Orders WHERE CustomerId = 5 AND OrderId = 203"

# 4. Verify DLQ Failure: Should return 0, confirming the row migration failed
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="SELECT COUNT(*) FROM Orders WHERE CustomerId = 99999 AND OrderId = 502"
```

預期輸出內容：

```
CustomerId: 4
OrderId: 501
OrderValue: 99.99
OrderSource: MobileApp

CustomerId: 5
OrderId: 202
OrderValue: 1500
OrderSource: WebStore

0
0
```

注意：如果任何查詢未顯示預期結果，請稍候片刻再試一次，因為串流工作人員可能仍在處理佇列。

B. 檢查 DLQ 中的「Intentional Failure」：

由於 `CustomerId = 99999` 在 `Customers` 資料表中沒有父項，因此應該會遭到 Spanner 拒絕，並由 Dataflow 安全地傳送至 DLQ。

1. 前往 Google Cloud 控制台的「Cloud Storage」。
2. 前往 bucket 並開啟 `live-migration/dlq/severe/` 資料夾。
3. 您應該會看到新產生的 JSON 檔案。點選即可檢查內容。您會看到 `CustomerId = 99999` 的詳細資料和特定的 Spanner 錯誤訊息：`NOT_FOUND: Parent row for row [99999,502,WebStore] in table Orders is missing. Row cannot be written."`

4. 如要重試即時遷移 DLQ 錯誤，請執行資料流範本並設定 `runMode=retryDLQ`。

5. 處理 DLQ 錯誤

`severe/` 目錄中的錯誤需要手動介入。請修正資料問題，然後重新處理失敗的事件。

A. 修正來源中的資料

發生錯誤的原因是缺少上層客戶記錄 `CustomerId = 99999`。現在要將其插入來源 MySQL 資料庫。

再次透過 SSH 連線至 MySQL 執行個體：

```
gcloud compute ssh mysql-physical-1 --zone=$ZONE
```

使用 `sudo mysql` 登入 MySQL，並將缺少的父項資料列插入 `shard1_db`：

```
USE shard1_db;

INSERT INTO Customers (CustomerId, CustomerName, CreditLimit, LegacyRegion) VALUES
(99999, 'DLQ Parent Holder', 5000.00, 'NORTH_AMERICA');

EXIT;
```

輸入 **exit** 以返回 Cloud Shell。

B. 執行 retryDLQ Dataflow 工作

如要重新處理 `severe/` DLQ 中的事件，請啟動相同的 Dataflow 範本，但須處於 `retryDLQ` 模式。這個模式會從 `deadLetterQueueDirectory/severe` 路徑讀取資料，透過自訂轉換重新執行資料，然後套用至 Spanner。

以 `retryDLQ` 模式啟動工作：

```
export JOB_NAME_RETRY="mysql-sharded-cdc-retry-$(date +%Y%m%d-%H%M%S)"

gcloud dataflow flex-template run $JOB_NAME_RETRY \
  --project=$PROJECT_ID \
  --region=$REGION \
  --worker-machine-type=n2-highmem-8 \
  --template-file-gcs-location="gs://dataflow-templates-${REGION}/latest/flex/Cloud_Datastream_to_Spanner" \
  --parameters \
  instanceId="$SPANNER_INSTANCE_NAME",\
  databaseId="$SPANNER_DATABASE_NAME",\
  projectId="$PROJECT_ID",\
  runMode="retryDLQ",\
  deadLetterQueueDirectory="$DLQ_DIR_CDC",\
  datastreamSourceType="mysql",\
  transformationJarPath=$CUSTOM_JAR_PATH,\
  transformationClassName="com.custom.CustomTransformationFetcher",\
  schemaOverridesFilePath=$GCS_OVERRIDES_PATH,\
  shardingContextFilePath=$GCS_LIVE_SHARDING_PATH
```

重試的 Key 參數異動

- `runMode="retryDLQ"`：告知範本從 `severe DLQ` 目錄讀取。
- 移除 `gcsPubSubSubscription`：我們不會從即時 Datastream GCS bucket 讀取資料，因此不需要這個項目。

監控重試程序：

與主要 CDC 管道類似，`retryDLQ` 是串流管道，會維持 `RUNNING` 狀態，直到手動取消為止。

1. 前往 `$JOB_NAME_RETRY` 的 Dataflow Job 頁面。
2. 在「指標」窗格下方，找出下列兩個計數器：
3. `elementsReconsumedFromDeadLetterQueue`：在擷取錯誤檔案時進行評估。
4. `Successful events`：將記錄寫入 Spanner 時會遞增。
5. 檢查 `severe/` 目錄是否有重複失敗的情況。
6. 「成功」事件的數量增加您想重試的項目數 (在本測試案例中為 1) 後，請前往下一個驗證步驟。

C. 驗證重試的資料

重試失敗的記錄 (可能需要一段時間才能成功) 後，請檢查 Spanner，確認子項資料列是否已成功遷移：

```
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \  
  --instance=$SPANNER_INSTANCE_NAME \  
  --sql="SELECT CustomerId, OrderId, OrderValue, OrderSource FROM Orders WHERE CustomerId = 99999 AND OrderId = 502"
```

現在您應該會看到以下資料列：

```
CustomerId: 99999  
OrderId: 502  
OrderValue: 50  
OrderSource: WebStore
```

此外，也請檢查 GCS 中的 `$DLQ_DIR_CDC/severe/` 資料夾。處理過的檔案應已移動或刪除，表示重新處理成功。

11. 設定反向複製 (Spanner 至 MySQL)

如要處理可能需要復原或在過渡期間讓原始 MySQL 資料庫與 Spanner 同步的情況，可以設定反向複製。

這個管道會使用 Spanner 變更串流，擷取 Spanner 中的即時修改內容。接著，這項服務會使用我們的自訂轉換 JAR，反向對應結構定義差異，並使用自訂分片 JAR，準確計算更新應寫回哪個實體 MySQL VM 和邏輯分片。

1. 建立 Spanner 變更串流

首先，您需要在 Spanner 資料庫中建立變更串流，追蹤 `Customers` 和 `Orders` 資料表的變更。

```
export CHANGE_STREAM_NAME="CustomersOrdersChangeStream"

gcloud spanner databases ddl update $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --ddl="CREATE CHANGE STREAM $CHANGE_STREAM_NAME FOR Customers, Orders"
```

這個變更串流現在會記錄指定資料表的所有資料修改。

2. 建立 Dataflow 中繼資料的 Spanner 資料庫

`Spanner to SourceDB` Dataflow 範本需要另外一個 Spanner 資料庫，才能儲存用於管理變更串流用量的中繼資料。

```
export SPANNER_METADATA_DB_NAME="migration-metadata-db"

gcloud spanner databases create $SPANNER_METADATA_DB_NAME \
  --instance=$SPANNER_INSTANCE_NAME
```

3. 準備 Dataflow 適用的 Cloud SQL 連線設定

Dataflow 範本需要 Cloud Storage 中的 JSON 檔案，內含目標 Cloud SQL 資料庫的連線詳細資料。

建立名為 `shard_config.json` 的本機檔案：

```
cat <<EOF > reverse-sharding.json
[
  {
    "logicalShardId": "shard0_db",
    "host": "${MYSQL_IP_1}",
    "port": "3306",
    "user": "datastream_user",
    "password": "complex_password_123",
    "dbName": "shard0_db"
  },
  {
    "logicalShardId": "shard1_db",
    "host": "${MYSQL_IP_1}",
    "port": "3306",
    "user": "datastream_user",
    "password": "complex_password_123",
    "dbName": "shard1_db"
  },
  {
    "logicalShardId": "shard2_db",
    "host": "${MYSQL_IP_2}",
    "port": "3306",
    "user": "datastream_user",
    "password": "complex_password_123",
    "dbName": "shard2_db"
  },
  {
    "logicalShardId": "shard3_db",
    "host": "${MYSQL_IP_2}",
    "port": "3306",
    "user": "datastream_user",
    "password": "complex_password_123",
    "dbName": "shard3_db"
  }
]
EOF
```

將這個檔案上傳至 GCS bucket：

```
export GCS_REVERSE_SHARDING_PATH="gs://${BUCKET_NAME}/config/reverse-sharding.json"
gcloud storage cp reverse-sharding.json $GCS_REVERSE_SHARDING_PATH
```

4. 執行反向複製 Dataflow 工作

使用 `Spanner_to_SourceDb` Flex 範本啟動 Dataflow 工作。

```

export JOB_NAME_REVERSE="spanner-sharded-reverse-to-mysql-$(date +%Y%m%d-%H%M%S)"
export DLQ_DIR_REVERSE="gs://${BUCKET_NAME}/reverse-replication"

gcloud dataflow flex-template run $JOB_NAME_REVERSE \
  --project=$PROJECT_ID \
  --region=$REGION \
  --worker-machine-type=n2-highmem-8 \
  --max-workers=2 \
  --num-workers=1 \
  --additional-experiments=use_runner_v2 \
  --template-file-gcs-location="gs://dataflow-templates-${REGION}/latest/flex/Spanner_to_SourceDb" \
  --parameters \
  changeStreamName="$CHANGE_STREAM_NAME",\
  instanceId="$SPANNER_INSTANCE_NAME",\
  databaseId="$SPANNER_DATABASE_NAME",\
  spannerProjectId="$PROJECT_ID",\
  metadataInstance="$SPANNER_INSTANCE_NAME",\
  metadataDatabase="$SPANNER_METADATA_DB_NAME",\
  sourceShardsFilePath="$GCS_REVERSE_SHARDING_PATH",\
  transformationJarPath=$CUSTOM_JAR_PATH,\
  transformationClassName="com.custom.CustomTransformationFetcher",\
  shardingCustomJarPath=$CUSTOM_JAR_PATH,\
  shardingCustomClassName="com.custom.CustomShardIdFetcher",\
  schemaOverridesFilePath=$GCS_OVERRIDES_PATH,\
  deadLetterQueueDirectory=$DLQ_DIR_REVERSE

```

重要參數

- `changeStreamName`：要讀取的 Spanner 變更串流名稱。
- `metadataInstance`, `metadataDatabase`：Spanner 執行個體/資料庫，用於儲存連接器使用的中繼資料，以控管變更串流 API 資料的用量。
- `sourceShardsFilePath`：`shard_config.json` 的 GCS 路徑。
- `filtrationMode`：指定如何根據條件捨棄特定記錄。預設為 `forward_migration` (篩選使用轉送遷移管道寫入的記錄)
- `shardingCustomJarPath`：先前建構的已編譯 Java JAR 檔案 GCS 路徑。
- `shardingCustomClassName`：完整類別名稱 (`com.custom.CustomShardIdFetcher`)，用於執行自訂 `%4` 模數運算，動態決定應接收記錄的邏輯分片。

網路注意事項：Dataflow 工作站會使用 `shard_config.json` 中指定的公開 IP 位址連線至 Cloud SQL 執行個體。由於 Cloud SQL 執行個體「授權網路」中的 `0.0.0.0/0` 項目，系統允許建立此連線。

在 [Dataflow Jobs 控制台](#) 中監控工作啟動情形。

5. 注入 Spanner 資料並觸發有意造成的失敗

等待 Dataflow 工作進入 `Running` 狀態 (這可能需要約 5 分鐘)。接著，我們將整套查詢 (`INSERT`、`UPDATE`、`DELETE`) 直接執行到 Spanner 中，並刻意發生失敗，以測試反向 DLQ。

在 Cloud Shell 中執行下列指令：

```
# All these operations are done on rows mapping to shard0_db for convenience

# Valid INSERT: Insert parent row in Customers
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="INSERT INTO Customers (CustomerId, CustomerName, CreditLimit, LoyaltyTier) VALUES
(88, 'Reverse Tester', 5000, 'GOLD_TIER')"
```

1. Valid INSERT (Orders): 'WebStore' transformed to 'WebStore_v1'

```
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="INSERT INTO Orders (CustomerId, OrderId, OrderValue, OrderSource) VALUES (88,
9001, 150.00, 'WebStore')"
```

2. Valid UPDATE (Orders)

```
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="UPDATE Orders SET OrderValue = 200.00 WHERE CustomerId = 16 AND OrderId = 105 AND
OrderSource = 'Partner'"
```

3. Valid DELETE (Orders)

```
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="DELETE FROM Orders WHERE CustomerId = 12 AND OrderId = 104 AND OrderSource =
'WebStore'"
```

4. INVALID Insert- DLQ Test: CreditLimit=500 will fail check constraint of >1000 at source

```
gcloud spanner databases execute-sql $SPANNER_DATABASE_NAME \
  --instance=$SPANNER_INSTANCE_NAME \
  --sql="INSERT INTO Customers (CustomerId, CustomerName, CreditLimit, LoyaltyTier) VALUES
(44, 'DLQ Test Customer', 500, 'GOLD_TIER')"
```

6. 驗證反向複寫資料並檢查 DLQ

讓我們確認 Custom Sharding JAR 是否已在第一個實體 VM 上，成功將 `CustomerId 88` 路由至 `shard0_db`，以及 Custom Transformation JAR 是否已成功從區域中移除 `"_TIER"`。

A. 在 MySQL 中驗證有效記錄：

透過 SSH 連入第一個實體分片：

```
gcloud compute ssh mysql-physical-1 --zone=$ZONE
```

登入 MySQL 並查詢 `shard0_db`：

```

sudo mysql
USE shard0_db;

-- 1. Verify INSERT: Row migrated with transformed LegacyOrderSystem
SELECT CustomerId, OrderId, OrderValue, LegacyOrderSystem
FROM Orders
WHERE CustomerId = 88 AND OrderId = 9001;

-- 2. Verify UPDATE: The OrderValue should now be updated to 200.00.
SELECT CustomerId, OrderId, OrderValue, LegacyOrderSystem
FROM Orders
WHERE CustomerId = 16 AND OrderId = 105;

-- 3. Verify DELETE: Returns 0 rows, confirming the order was successfully deleted from
MySQL.
SELECT CustomerId, OrderId
FROM Orders
WHERE CustomerId = 12 AND OrderId = 104;

-- 4. Verify failed replication - this should be in DLQ as CreditLimit < 1000 and will fail
stricter check constraint at source
SELECT CustomerId, CustomerName, CreditLimit, LegacyRegion
FROM Customers
WHERE CustomerId = 44;

EXIT;

```

Cloud SQL 中的預期輸出內容應反映 Spanner 中所做的變更。

```

+-----+-----+-----+-----+
| CustomerId | OrderId | OrderValue | LegacyOrderSystem |
+-----+-----+-----+-----+
|          88 |      9001 |       150.00 | Webstore_v1      |
+-----+-----+-----+-----+

+-----+-----+-----+-----+
| CustomerId | OrderId | OrderValue | LegacyOrderSystem |
+-----+-----+-----+-----+
|          16 |       105 |       200.00 | Partner_v1       |
+-----+-----+-----+-----+

Empty set (0.00 sec)
Empty set (0.00 sec)

```

類型

exit

返回 Cloud Shell。

這會確認反向複製管道是否正常運作，並將 Spanner 中的變更同步回 Cloud SQL。

B. 檢查 DLQ 中的刻意失敗

由於新的 `Customers` 記錄的 `CreditLimit` 為 500 (違反我們在來源 MySQL 資料庫中定義的嚴格 `> 1000` 檢查限制)，Dataflow 安全地偵測到錯誤。

1. 前往 Google Cloud 控制台的「Cloud Storage」。
 2. 前往 bucket 並開啟 `dlq/severe/` 資料夾。
 3. 開啟 JSON 檔案，查看遭拒的 `Customers` 記錄和確切的檢查限制違規錯誤。
 4. 如要重試反向複製 DLQ 錯誤，請執行設有 `runMode=retryDLQ` 的 Dataflow 範本。
-

12. 清除資源

如要避免系統向您的 Google Cloud 帳戶收取額外費用，請刪除本程式碼研究室建立的資源。

設定環境變數 (如有需要)

如果 Cloud Shell 工作階段逾時，或是您開啟了新的終端機，請先重新匯出環境變數，再執行清除指令。

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1" # Or your preferred region
export ZONE="us-central1-a" # Or a zone within your selected region
export SPANNER_INSTANCE_NAME="target-spanner-instance"
export SPANNER_DATABASE_NAME="sharded-target-db"
export SPANNER_CONFIG="regional-{$REGION}"
export BUCKET_NAME="migration-{$PROJECT_ID}-bucket"
export MYSQL_IP_1=$(gcloud compute instances describe mysql-physical-1 --zone=$ZONE --
format='get(networkInterfaces[0].accessConfigs[0].natIP)')
export MYSQL_IP_2=$(gcloud compute instances describe mysql-physical-2 --zone=$ZONE --
format='get(networkInterfaces[0].accessConfigs[0].natIP)')
export SQL_CP_NAME_1="mysql-src-cp-1"
export SQL_CP_NAME_2="mysql-src-cp-2"
export GCS_CP_NAME="gcs-dest-cp"
export STREAM_NAME_1="mysql-to-spanner-stream-1"
export GCS_STREAM_PATH_1="data/{$STREAM_NAME_1}"
export STREAM_NAME_2="mysql-to-spanner-stream-2"
export GCS_STREAM_PATH_2="data/{$STREAM_NAME_2}"
export PUBSUB_TOPIC="datastream-gcs-updates"
export PUBSUB_SUBSCRIPTION="datastream-gcs-sub"
export CUSTOM_JAR_PATH="gs://{$BUCKET_NAME}/custom-logic/spanner-custom-shard-1.0.jar"
export OVERRIDES_FILE="spanner_overrides.json"
export GCS_OVERRIDES_PATH="gs://{$BUCKET_NAME}/config/{$OVERRIDES_FILE}"
export GCS_SHARDING_PATH="gs://{$BUCKET_NAME}/config/sharding.json"
export OUTPUT_DIR="gs://{$BUCKET_NAME}/bulk-migration"
export GCS_LIVE_SHARDING_PATH="gs://{$BUCKET_NAME}/config/live-sharding.json"
export DLQ_DIR_CDC="gs://{$BUCKET_NAME}/live-migration"
export CHANGE_STREAM_NAME="CustomersOrdersChangeStream"
export SPANNER_METADATA_DB_NAME="migration-metadata-db"
export GCS_REVERSE_SHARDING_PATH="gs://{$BUCKET_NAME}/config/reverse-sharding.json"
export DLQ_DIR_REVERSE="gs://{$BUCKET_NAME}/reverse-replication"
```

停止 Dataflow 串流工作

列出工作，找出執行中 Dataflow 工作的 Job ID。據此匯出 `JOB_ID_CDC` 和 `JOB_ID_REVERSE`。

```
gcloud dataflow jobs list --region=$REGION --filter="state=Running"
export JOB_ID_CDC=<PASTE_JOB_ID_HERE>
export JOB_ID_CDC_RETRY=<PASTE_JOB_ID_HERE>
export JOB_ID_REVERSE=<PASTE_JOB_ID_HERE>
```

取消 `Datastream to Spanner` (即時遷移) 工作及其重試工作：

```
gcloud dataflow jobs cancel $JOB_ID_CDC --region=$REGION --project=$PROJECT_ID

gcloud dataflow jobs cancel $JOB_ID_CDC_RETRY --region=$REGION --project=$PROJECT_ID
```

取消 `Spanner to Cloud SQL` (反向複製) 工作：

```
gcloud dataflow jobs cancel $JOB_ID_REVERSE --region=$REGION --project=$PROJECT_ID
```

刪除 Datastream 來源

停止並刪除串流：

```
gcloud datastream streams update $STREAM_NAME_1 \
  --location=$REGION --state=PAUSED --project=$PROJECT_ID
# Wait a moment for the stream to pause
gcloud datastream streams delete $STREAM_NAME_1 \
  --location=$REGION --project=$PROJECT_ID --quiet

gcloud datastream streams update $STREAM_NAME_2 \
  --location=$REGION --state=PAUSED --project=$PROJECT_ID
# Wait a moment for the stream to pause
gcloud datastream streams delete $STREAM_NAME_2 \
  --location=$REGION --project=$PROJECT_ID --quiet

# Delete Connection Profiles
gcloud datastream connection-profiles delete $SQL_CP_NAME_1 \
  --location=$REGION --project=$PROJECT_ID --quiet
gcloud datastream connection-profiles delete $SQL_CP_NAME_2 \
  --location=$REGION --project=$PROJECT_ID --quiet
gcloud datastream connection-profiles delete $GCS_CP_NAME \
  --location=$REGION --project=$PROJECT_ID --quiet
```

刪除來源 MySQL VM (Compute Engine)

刪除模擬地端部署 MySQL 實體分片的兩個 Compute Engine 執行個體。

```
gcloud compute instances delete mysql-physical-1 mysql-physical-2 --zone=$ZONE --quiet
```

刪除防火牆規則

移除為允許 SSH 存取和 Datastream 連線至 VM 而建立的網路防火牆規則。(注意：如果您在本程式碼研究室稍早為防火牆規則使用不同名稱，請在此處調整)。

```
gcloud compute firewall-rules delete allow-ssh-iap --quiet
gcloud compute firewall-rules delete allow-mysql-datastream --quiet
```

刪除 Pub/Sub 資源

刪除訂閱項目：

```
gcloud pubsub subscriptions delete $PUBSUB_SUBSCRIPTION \  
  --project=$PROJECT_ID --quiet
```

刪除主題：

```
gcloud pubsub topics delete $PUBSUB_TOPIC \  
  --project=$PROJECT_ID --quiet
```

刪除 Cloud Spanner 執行個體

刪除 Cloud Spanner 執行個體 (這會自動刪除其中的 `sharded-target-db` 和 `migration-metadata-db` 資料庫)。

```
gcloud spanner instances delete $SPANNER_INSTANCE_NAME \  
  --project=$PROJECT_ID --quiet
```

刪除 GCS Bucket 和內容

最後，刪除存放 Datastream 檔案、Dataflow 設定和 Dead Letter Queues 的 Cloud Storage bucket。 `rm -r` 指令會以遞迴方式刪除 bucket 和所有內容。

注意：如果您打算稍後檢查、排解或重試無效信件佇列 (DLQ) 中的失敗記錄，請**不要**刪除這個值區！

```
gcloud storage rm --recursive gs://{BUCKET_NAME}
```

刪除本機 Cloud Shell 檔案

如要清理在本程式碼實驗室期間，Cloud Shell 中產生的本機檔案和目錄，請執行下列指令：

```
# Remove the JSON configuration files  
rm -f sharding.json live-sharding.json reverse-sharding.json spanner_overrides.json  
  
# Remove the cloned Google Cloud DataflowTemplates repository  
rm -rf DataflowTemplates
```